



UNIVERSIDADE FEDERAL DA BAHIA
ESCOLA POLITÉCNICA
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E DE
COMPUTAÇÃO
ENGG56 - PROJETO DE CIRCUITOS INTEGRADOS DIGITAIS



DANIEL DO NASCIMENTO GOMES
EDUARDO VITOR DOS SANTOS SILVA E SILVA
THIAGO LUIGI MARINHO DE MELLO

PROJETO SEMESTRAL

Salvador - Bahia

Janeiro de 2026

DANIEL DO NASCIMENTO GOMES
EDUARDO VITOR DOS SANTOS SILVA E SILVA
THIAGO LUIGI MARINHO DE MELLO

PROJETO SEMESTRAL

Relatório apresentado ao professor
Wagner L. A. de Oliveira como requisito
para obtenção de créditos na disciplina de
Projeto De Circuitos Integrados Digitais.

Salvador - Bahia

Janeiro de 2026

SUMÁRIO

1. INTRODUÇÃO	5
2. OBJETIVOS	6
2.1. GERAL	6
2.2. ESPECÍFICOS	6
3. PROJETOS	7
3.1. CONTROLE REMOTO	7
3.1.1. Descrição do problema	7
3.1.2. Fundamentação Teórica e Solução Adotada	7
3.1.2.2. Definição da Taxa de Transmissão e Estratégia de Sincronização	8
3.1.3. Arquitetura do Sistema e Máquina de Estados (FSM)	8
3.1.4. Implementação em Verilog	9
3.1.5. Time Constraints	10
3.1.6. Simulação (Testbench)	11
3.2. ACUMULADOR COM MEMÓRIA EXTERNA	14
3.2.1. Descrição do problema	14
3.2.2. Arquitetura do módulo	15
3.2.3. Módulo Acumulador	17
3.2.4. FSM de Controle	18
3.2.5. Módulo TOP	19
3.2.6. Memória	20
3.2.7. Time Constraints	21
3.2.8. Simulação (Testbench)	22
3.2.8.1. Testbench do Acumulador (tb_Acumulador.v)	24
3.2.8.2. Testbench da FSM (tb_FSM.v)	26
3.2.8.3. Testbench do Sistema Completo (tb_TOP.v)	28
3.3. CODIFICAÇÃO DE VÍDEO MPEG	32
3.3.1. Descrição do problema	32
3.3.2. Arquitetura do módulo	33
3.3.3. Máquina de MOORE	34
3.3.4. FSM de Controle	35
3.3.5. Time Constraints	36
3.3.6. Simulação (Testbench)	37
3.4. SINALEIRA	40
3.4.1. Descrição do problema	40
3.4.2. Simplificação lógica	40
3.4.3. Implementação em Verilog	42
3.4.4. Time Constraints e Pin Assignments	42
3.4.5. Simulação (Testbench)	43
4. CONCLUSÃO	44

5. ANEXOS	44
6. REFERÊNCIAS	45

1. INTRODUÇÃO

Neste trabalho, buscou-se aplicar de forma prática os conhecimentos adquiridos ao longo do semestre, envolvendo o projeto, a implementação e a validação de circuitos digitais por meio da linguagem de descrição de hardware Verilog.

Ao longo do projeto, foram desenvolvidos quatro sistemas digitais distintos, cada um abordando diferentes conceitos fundamentais da disciplina, como lógica combinacional, máquinas de estados finitos, controle de memória externa, sincronização por clock e geração de sinais de controle. Todos os projetos foram implementados utilizando o software Quartus Prime Lite, versão 20.1.1, da Intel Corporation, e tiveram como objetivo final a possível implementação na placa de desenvolvimento DE2-115.

Além da implementação em Verilog, foram desenvolvidos testbenches específicos para cada módulo, com o intuito de avaliar o correto funcionamento dos circuitos propostos. As simulações funcionais foram realizadas no ambiente ModelSim, permitindo a verificação detalhada do comportamento dos sinais, bem como a validação das funcionalidades exigidas em cada atividade. Adicionalmente, os projetos passaram pelo processo de síntese no Quartus, respeitando as restrições temporais estabelecidas, conforme orientações da disciplina.

Os quatro projetos desenvolvidos contemplam aplicações clássicas de sistemas digitais. O primeiro consiste em um módulo capaz de realizar a leitura e o reconhecimento de informações provenientes de um controle remoto por infravermelho, validando os dados recebidos e identificando corretamente a tecla pressionada. O segundo projeto refere-se à implementação de um acumulador conectado a uma memória externa do tipo SRAM, no qual uma máquina de estados finitos controla a leitura sequencial de conjuntos específicos de dados, a acumulação dos valores lidos e a gravação dos resultados em posições determinadas da memória, seguindo um padrão de endereçamento previamente definido.

O terceiro projeto aborda o controle de operações relacionadas à codificação de vídeo, por meio de uma máquina de estados finitos responsável pela geração de sinais de controle para módulos auxiliares utilizados no processo de compressão MPEG de um bloco de pixels. Por fim, o quarto projeto consiste no desenvolvimento de um circuito lógico combinacional otimizado, baseado em mapas de Karnaugh, para o controle de um sistema de semáforo a partir de sensores de detecção de veículos.

Dessa forma, o trabalho proporciona uma visão integrada dos principais conceitos estudados na disciplina, permitindo a consolidação do aprendizado teórico por meio da aplicação prática em

projetos reais de circuitos digitais, desde a modelagem até a simulação e validação dos sistemas desenvolvidos.

2. OBJETIVOS

2.1. GERAL

O objetivo geral deste projeto é aplicar os conceitos fundamentais de Projeto de Circuitos Integrados Digitais, por meio do desenvolvimento, implementação e validação de sistemas digitais descritos em linguagem Verilog, contemplando o uso de lógica combinacional, máquinas de estados finitos, controle de memória externa e geração de sinais de controle, bem como a realização de simulações funcionais e síntese dos circuitos desenvolvidos.

2.2. ESPECÍFICOS

Os objetivos específicos deste projeto são:

- Projetar e implementar um módulo em Verilog capaz de identificar e validar sinais provenientes de um controle remoto infravermelho, reconhecendo corretamente a tecla pressionada e indicando a recepção válida dos dados;
- Desenvolver e implementar um sistema de somatório de dados baseado em uma memória externa do tipo SRAM, no qual uma máquina de estados finitos gerencia o processo de leitura de conjuntos específicos de dados, a acumulação dos valores lidos e a posterior gravação dos resultados em posições determinadas da memória, seguindo um padrão de endereçamento previamente estabelecido;
- Implementar uma máquina de estados finitos responsável pela geração de sinais de controle para módulos auxiliares envolvidos no processo de codificação de vídeo MPEG, coordenando operações de leitura de coeficientes e controle de cálculo;
- Projetar um circuito lógico combinacional otimizado, com base em mapas de Karnaugh, para o controle de um sistema de semáforo a partir de sensores de detecção de veículos, garantindo o correto funcionamento e evitando estados de conflito;
- Validar todos os módulos desenvolvidos por meio de simulações funcionais utilizando testbenches no ambiente ModelSim, bem como realizar a síntese dos circuitos no Quartus Prime, respeitando as restrições temporais estabelecidas.

3. PROJETOS

3.1. CONTROLE REMOTO

3.1.1. Descrição do problema

O objetivo desta etapa inicial do projeto é o desenvolvimento de um módulo chamado RemoteController. Esse módulo atua como uma interface de decodificação para sinais de controle remoto transmitidos via infravermelho. O sistema foi projetado para operar de forma síncrona em relação ao clock, porém assíncrona em relação ao sinal de controle, monitorando continuamente a entrada e verificando as informações recebidas.

O problema impõe a validação da integridade dos dados recebidos. O módulo deve capturar um frame composto por um bit de lead, seguido de 32 bits de payload e, por fim, um bit indicando o término do dado. Em seguida, deve verificar se os campos correspondentes ao código da tecla (Key Code) e ao seu inverso (Inv Key Code) são complementares. Apenas em caso de validação positiva, a saída Ready deve ser ativada por exatamente três pulsos de clock, disponibilizando o código da tecla decodificada na saída Tecla.

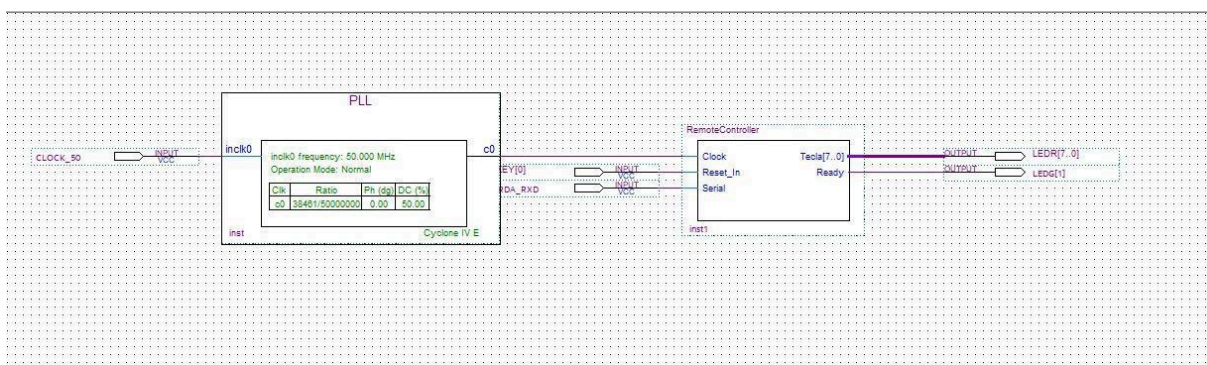


Figura 1- Diagrama de conexões do módulo RemoteController

3.1.2. Fundamentação Teórica e Solução Adotada

3.1.2.1. Protocolo NEC e Adaptações

Por meio da análise da imagem disponibilizada e de pesquisas realizadas, verificou-se que o protocolo utilizado é semelhante ao protocolo NEC, amplamente empregado em sistemas eletrônicos

Embora o protocolo NEC original utilize codificação por distância de pulso (na qual os bits '0' e '1' possuem durações diferentes), a especificação do problema simplificou o método de transmissão e determinou que os bits '0' e '1' tenham a mesma largura temporal, sendo codificados diretamente como nível lógico baixo e alto, respectivamente.

3.1.2.2. Definição da Taxa de Transmissão e Estratégia de Sincronização

Dada a ausência de especificação explícita da taxa de bits no enunciado, tornou-se necessário arbitrar uma frequência de operação para o sistema.

Considerou-se, inicialmente, a possibilidade de adotar como referência o protocolo NEC padrão, utilizando o intervalo entre duas bordas de descida do sinal como sendo de 2,25 ms. Caso esse período fosse adotado como a largura fixa do bit, a frequência de operação resultante seria extremamente baixa, exigindo, além de um PLL para geração de uma frequência reduzida, o uso adicional de contadores de pulso.

Optou-se, portanto, por utilizar como referência a portadora padrão de infravermelho, que é de 38 kHz. Assim, o período de cada bit foi definido como:

$$T = \frac{1}{38000} \simeq 26,31\mu S$$

Diferentemente de abordagens que utilizam amostragem com clocks significativamente mais rápidos e contadores internos, a estratégia adotada consistiu em projetar o sistema com um clock principal cujo período é próximo ao da taxa de transmissão. Essa escolha elimina a necessidade do uso de contadores de pulso dentro da máquina de estados. Como a frequência do clock é muito próxima à frequência estimada de transmissão, o sistema realiza a captura dos dados diretamente na borda do clock, assumindo que o sincronismo é mantido ao longo do frame curto de 32 bits.

Contudo, a captura direta de um sinal assíncrono por um sistema síncrono pode ocasionar metaestabilidade. Para reduzir esse risco, foi implementado um sincronizador composto por dois flip-flops.

3.1.3. Arquitetura do Sistema e Máquina de Estados (FSM)

Para a implementação do RemoteController, o sistema foi dividido em uma máquina de estados que coordena a captura e validação dos dados visto na Figura 2.

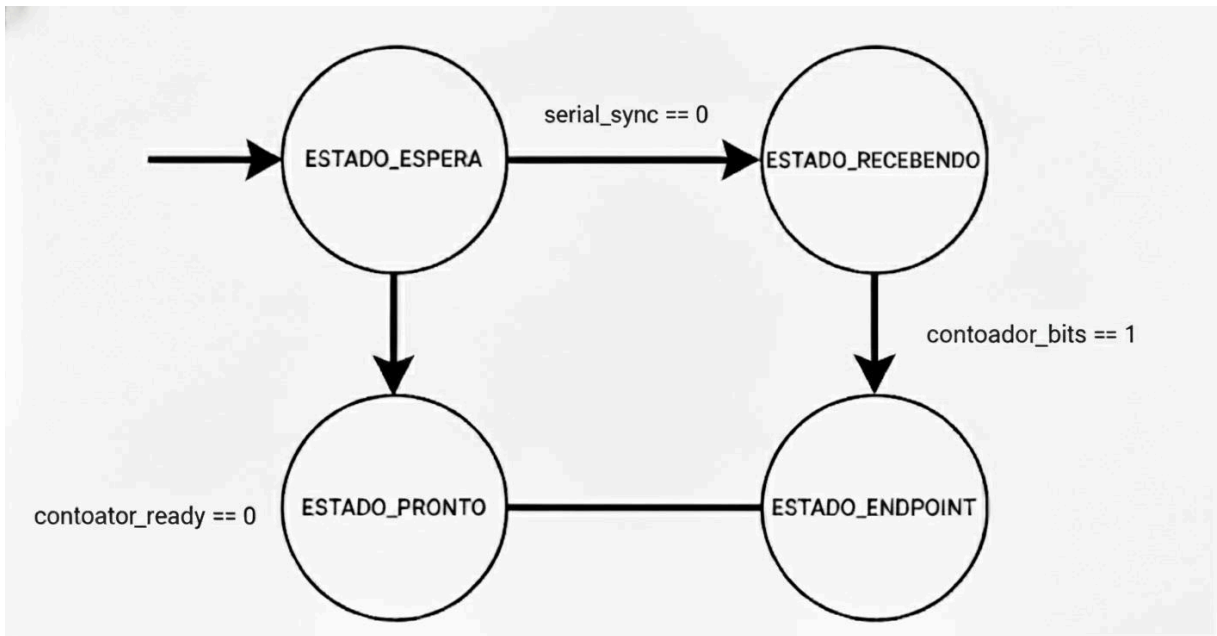


Figura 2 - Diagrama da Máquina de Estados do RemoteController.

O diagrama de estados pode ser descrito da seguinte forma:

- **ESTADO_ESPERA**: O sistema aguarda o envio do Lead Code.
- **ESTADO_RECEBENDO**: O sistema realiza o deslocamento (shift) dos bits recebidos para um registrador interno de 32 bits. Um contador auxiliar monitora o número de bits recebidos até completar o pacote.
- **ESTADO_ENDPOINT**: Estado em que o sistema recebe o endpoint e também inicializa o contador responsável pelos três pulsos da saída.
- **ESTADO_PRONTO**: Caso a validação seja bem-sucedida, o sinal Ready é ativado, e um contador de pulsos garante a duração de 3 ciclos de clock antes de retornar ao **ESTADO_ESPERA**.

3.1.4. Implementação em Verilog

```

always @(posedge Clock) begin
    rst_ff1 <= Reset_In;
    rst_sync <= rst_ff1;

    serial_ff1 <= Serial;
    serial_sync <= serial_ff1;
end
  
```

O código implementa um sincronizador de dois estágios para as entradas Reset_In e Serial, utilizando dois flip-flops em cascata para cada sinal.

3.1.5. Time Constraints

Ao executarmos o Timing Analyzer, observamos que as configurações padrão do projeto não eram adequadas para a nossa aplicação. Por padrão, quando não há restrições explícitas, o Quartus assume uma frequência de clock extremamente alta para análise de setup e hold.

Para corrigir isso e garantir uma análise de temporização adequada, aplicamos restrições com os seguintes parâmetros:

- **create_clock**: Define a origem do clock com frequência de 50 MHz.
- **create_generated_clock**: Define o período operacional do nosso sistema em 26,31 μ s.
- **set_clock_uncertainty**: Define uma margem de incerteza de 0,020 ns para cobrir eventuais oscilações do sinal.
- **set_input_delay / set_output_delay**: Define o atraso de propagação de 2,6 ns.

Para validar as condições ambientais, a verificação temporal foi realizada considerando os cenários extremos de operação. Conforme consta nas imagens dos relatórios do Timing Analyzer (Figuras 3, 4 e 5), foram examinados os modelos Slow 1200 mV 85°C, Slow 1200 mV 0°C e Fast 1200 mV 0°C. Essa verificação cruzada assegurou as margens de Setup Time e Hold Time.

	Clock	Slack	End Point TNS
1	inst altpll_component auto_generated pll1 clk[0]	25989.521	0.000

Figura 3 - Relatório de *Setup Summary* para o modelo Slow 1200mV 85°C

	Clock	Slack	End Point TNS
1	inst[altpll_component]auto_generated[pll1]clk[0]	0.353	0.000

Figura 4 - Relatório de *Hold Summary* para o modelo Slow 1200mV 0°C

	Clock	Slack	End Point TNS
1	inst[altpll_component]auto_generated[pll1]clk[0]	0.181	0.000

Figura 5 - Relatório de *Hold Summary* para o modelo Fast 1200mV 0°C

3.1.6. Simulação (Testbench)

Para validar o funcionamento do módulo RemoteController, utilizamos uma simulação através de um Testbench denominado Tb_RemoteController. O objetivo foi simular o comportamento físico de um controle remoto IR e verificar se o RemoteController é capaz de capturar, validar e decodificar os sinais corretamente.

O testbench não se limitou a aplicar sinais ideais: o clock do transmissor foi implementado com um período levemente distinto em relação ao do receptor, simulando condições mais realistas. A alteração da tecla simulada resume-se à modificação de uma linha no código-fonte:

```
// Envia: Endereço (0x0000), Comando (0x10) e Comando Inverso (0xEF)
enviar_pacote_customizado(16'h0000, 8'h10, 8'hEF);
```

A metodologia de verificação foi dividida em três testes distintos:

- **Transmissão Padrão:** Validar o fluxo ideal de funcionamento. Enviamos o código correspondente à tecla 'C' (0x10) com o payload correto. O sistema comportou-se conforme o esperado, detectando o Start Bit, transitando do estado ESTADO_ESPERA para ESTADO_RECEBENDO e, após a recepção dos 32 bits, validando a integridade dos dados. Como pode ser observado na forma de onda da simulação (Figura 6), o sinal Ready foi elevado para nível lógico alto ao final do pacote, confirmando que o dado foi aceito e disponibilizado na saída.

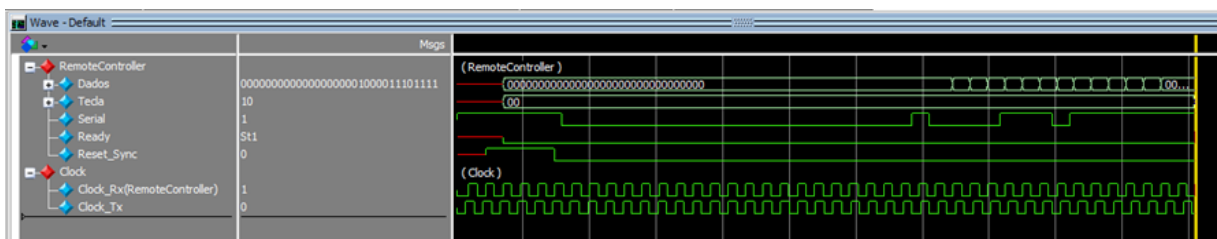


Figura 6 - Transmissão válida e ativação do sinal Ready

```
#
# --- ENVIANDO PACOTE MANUAL ---
# Custom: 0000 | Key: 10 | Check: ef
# -> Enviando Endpoint...
# Pacote enviado. Aguardando reacao do hardware...
#
# >>> READY DETECTADO! <<<
# Dados Hex: 000010ef (Tecla: 10)
# INTEGRIDADE: OK (Checksum valido).
# IDENTIFICACAO: Tecla C
# ** Note: $stop      : C:/ENGG56/RemoteController/Tb_RemoteController.v(159)
#    Time: 1110100 ns  Iteration: 1  Instance: /tb_RemoteController_ManualCheck
```

Figura 7 - Log do Console Exibindo a detecção da Tecla 'C'

- **Independência entre Decodificação do Protocolo e Identificação da Tecla:** Foi enviado um pacote válido contendo um código de tecla não mapeado na lista de comandos conhecidos do Testbench. O módulo de hardware validou e ativou o sinal **Ready** (como visto no detalhe da Figura 8), indicando que a transmissão elétrica foi bem-sucedida. Contudo, o console do Testbench sinalizou corretamente o alerta de "Tecla Desconhecida", demonstrando que o

sistema é capaz de receber novos códigos sem travar, deixando a interpretação da função para a camada superior.

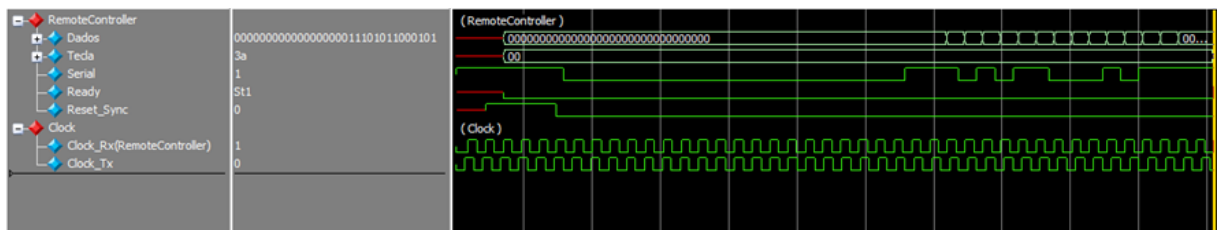


Figura 8 - Waveform confirmando a ativação do Ready

```
# --- ENVIANDO PACOTE MANUAL ---
# Custom: 0000 | Key: 3a | Check: c5
# -> Enviando Endpoint...
# Pacote enviado. Aguardando reacao do hardware...
#
# >>> READY DETECTADO! <<<
# Dados Hex: 00003ac5 (Tecla: 3a)
# INTEGRIDADE: OK (Checksum valido).
# IDENTIFICACAO: NAO RECONHECE A TECLA (Codigo 3a desconhecido)
# ** Note: $stop      : C:/ENGG56/RemoteController/Tb_RemoteController.v(159)
#   Time: 1110100 ns  Iteration: 1  Instance: /tb_RemoteController_ManualCheck
```

Figura 9 - Log indicando o recebimento de uma tecla válida, porém desconhecida.

- **Rejeição de Pacotes com Erros:** Foi enviado um pacote com falha de integridade, no qual o byte de verificação (checksum) não correspondia ao inverso do comando enviado. Como evidenciado na Figura 10, o módulo detectou a inconsistência e manteve o sinal **Ready** em nível baixo, garantindo que pacotes incorretos não fossem aceitos.

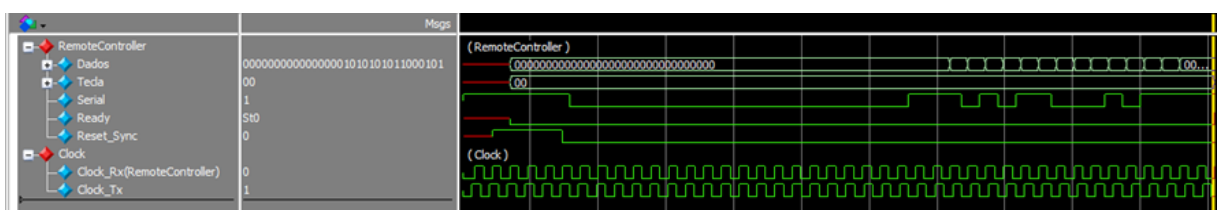


Figura 10 - Waveform mostrando que o sinal Ready permaneceu Baixo

```

#
# --- ENVIANDO PACOTE MANUAL ---
# Custom: 0000 | Key: aa | Check: c5
# -> Enviando Endpoint...
# Pacote enviado. Aguardando reacao do hardware...
#
# >>> TIMEOUT (Ready nao subiu) <<<
# ** Note: $stop      : C:/ENGG56/RemoteController/Tb_RemoteController.v(159)
#    Time: 1652560 ns  Iteration: 1  Instance: /tb_RemoteController_ManualCheck

```

Figura 11 - Log reportando a falha de integridade

3.2. ACUMULADOR COM MEMÓRIA EXTERNA

3.2.1. Descrição do problema

O segundo projeto do trabalho consiste na implementação de um sistema digital capaz de realizar somatórios sequenciais a partir de dados armazenados em uma memória externa do tipo SRAM. A memória possui profundidade de 32 endereços, cada um com largura de 16 bits. O sistema deve ser controlado por uma máquina de estados finitos (FSM) que gerencia todo o fluxo de operações de leitura, acumulação e escrita.

A especificação define que a FSM deve processar quatro conjuntos contíguos de endereços:

- Bloco 1: endereços 0 a 6
- Bloco 2: endereços 8 a 14
- Bloco 3: endereços 16 a 22
- Bloco 4: endereços 24 a 30

Para cada bloco, o sistema deve ler sequencialmente os valores armazenados nos sete primeiros endereços, acumulá-los em um somador, e gravar o resultado no oitavo endereço do bloco (respectivamente, endereços 7, 15, 23 e 31). Ao final do processamento dos quatro blocos, um sinal de saída denominado Ready deve ser ativado por exatamente um pulso de clock, indicando para um controlador externo que a operação foi concluída. Após isso, a FSM deve reiniciar o ciclo automaticamente.

O projeto foi implementado em Verilog possuindo três módulos principais: um acumulador, uma FSM de controle e um módulo TOP indicado pelo número 1 na Figura 12 e dentro dele possui o acumulador e o FSM como a Figura 13 indicado pela seta e faz a interface com a memória externa como pode ser observado nas Figuras 12 e 13. Além disso, foi necessário realizar simulações funcionais com testbenches específicos e aplicar restrições temporais adequadas para garantir o correto funcionamento quando implementado na placa DE2-115.

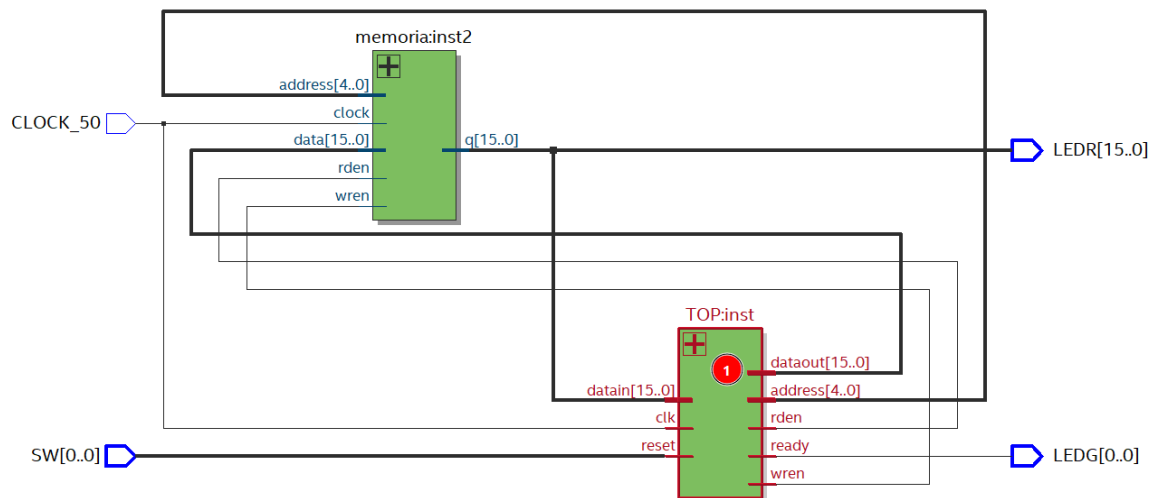


Figura 12 - Diagrama de blocos do circuito completo compacto

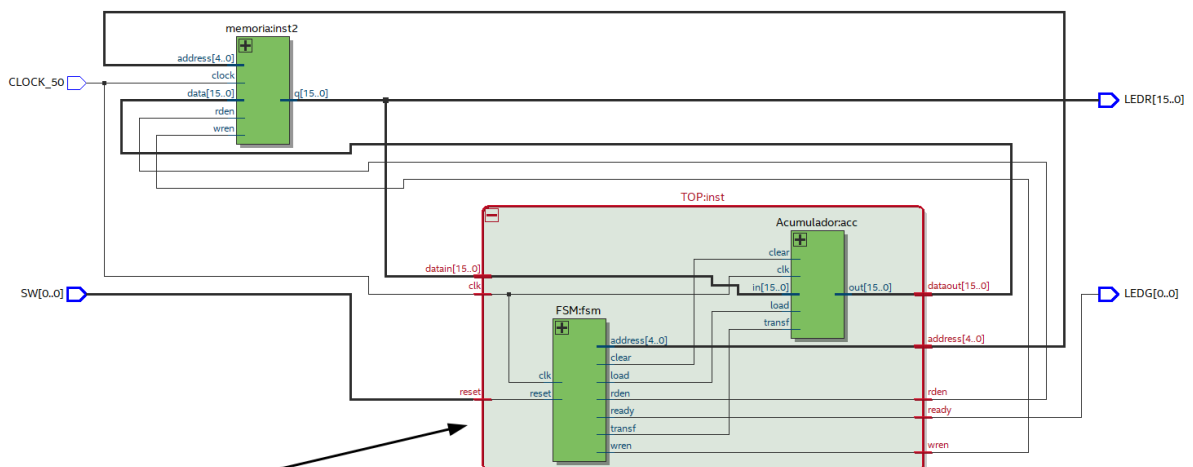


Figura 13 - Diagrama de blocos do circuito completo expandido

3.2.2. Arquitetura do módulo

A arquitetura do sistema foi estruturada em três níveis hierárquicos, conforme ilustrado na Figura 13 (Diagrama de blocos do circuito completo expandido). No nível mais alto, o módulo TOP serve como interface principal entre o sistema e os componentes externos, além de realizar a interligação com os pinos físicos da placa FPGA.

O sinal de clock do sistema é fornecido pelo oscilador da placa, conectado ao pino CLOCK_50 (PIN_Y2), responsável por sincronizar todas as operações internas. O sinal de reset é proveniente da chave SW[0], conectada ao pino PIN_AB28, permitindo a reinicialização manual do sistema. Esses sinais são aplicados diretamente às entradas clk e reset do módulo TOP.

O módulo TOP também recebe os dados lidos da memória externa através do barramento datain[15:0], enquanto fornece os sinais de controle address[4:0], rden e wren para acesso à memória. O sinal de conclusão do processamento, ready, é disponibilizado externamente e conectado ao LEDG[0], permitindo ao usuário visualizar quando a operação foi finalizada.

Internamente, o módulo TOP instancia dois submódulos principais: o Acumulador e a FSM (Máquina de Estados Finitos). O Acumulador é responsável por armazenar temporariamente os dados lidos da memória e realizar a soma acumulada, enquanto a FSM gera toda a sequência de controle necessária para coordenar as operações do acumulador e da memória externa.

A comunicação entre a FSM e o Acumulador é realizada por meio dos sinais load (captura do dado lido), transf (transferência do valor para a soma acumulada) e clear (zeragem do acumulador entre blocos). A FSM também controla diretamente os sinais rden e wren, assegurando o correto sincronismo das operações de leitura e escrita da memória SRAM.

A memória externa foi implementada utilizando o IP Core de memória do Quartus, configurado como uma RAM síncrona de porta única, com largura de dados de 16 bits e endereçamento de 5 bits (32 endereços). A memória é inicializada a partir de um arquivo RAM.mif, contendo valores predefinidos para fins de teste e validação do sistema.

Os dados de saída da memória (q[15:0]) são conectados diretamente aos LEDs vermelhos da placa (LEDR[15:0]), mapeados fisicamente aos pinos PIN_G19, PIN_F19, PIN_F21, PIN_F18, PIN_E18, PIN_J19, PIN_H19, PIN_J17, PIN_G17, PIN_J15, PIN_H16, PIN_J16, PIN_H17, PIN_F15 e PIN_G15, permitindo a visualização direta dos valores armazenados na memória durante a execução.

O controlador externo mencionado na especificação não foi implementado fisicamente; entretanto, sua interface lógica foi preservada por meio dos sinais clock, reset e ready, garantindo que o sistema permaneça compatível com a especificação original do projeto. O diagrama de conexões pode ser visualizado na Figura 14.

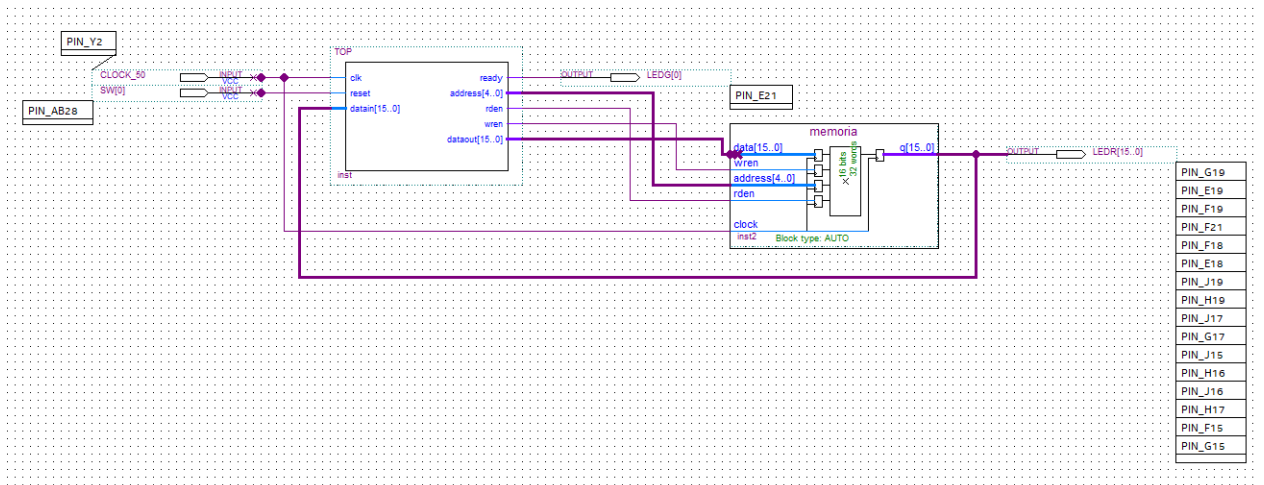


Figura 14 - Diagrama de blocos do circuito com os pinos

3.2.3. Módulo Acumulador

O módulo Acumulador foi projetado como um somador com registradores de entrada e saída. Sua implementação em Verilog segue o código apresentado no arquivo Acumulador.v. O módulo possui as seguintes portas:

- clk: sinal de clock para sincronização
- in[15:0]: barramento de entrada de dados (conectado à saída da memória)
- load: sinal que, quando ativo, captura o valor presente em in para um registrador interno (data_buffer)
- transf: sinal que, quando ativo, soma o conteúdo do data_buffer ao valor acumulado na saída out
- clear: sinal de reset assíncrono, que quando em nível baixo (0) zera tanto o data_buffer quanto o acumulador out
- out[15:0]: barramento de saída com o resultado acumulado

A lógica interna foi dividida em dois processos sensíveis à borda de descida do clock (negedge clk) e ao sinal clear. Esta escolha de usar negedge ajuda a evitar condições de corrida com a FSM, que normalmente opera na borda de subida.

No primeiro processo, quando clear está inativo (nível alto) e load está ativo, o valor de in é armazenado em data_buffer na borda de descida do clock. O sinal clear tem prioridade máxima: quando em nível baixo, data_buffer é imediatamente zerado, independentemente do estado de load.

No segundo processo, quando o clear está inativo e transf está ativo, o valor de data_buffer é somado ao registrador out, também na borda de descida do clock. Novamente, clear tem prioridade: quando ativo, zera imediatamente o acumulador out.

Esta implementação cria um pipeline de um ciclo de latência entre a captura do dado e sua soma ao acumulador. Quando load está ativo em um ciclo, o dado é capturado no data_buffer na borda de descida; no ciclo seguinte, se transf for ativado, este valor será somado ao acumulador.

A operação do acumulador segue portanto uma sequência:

1. Fase de captura: load = 1 → dado armazenado em data_buffer (no negedge)
2. Fase de acumulação: transf = 1 → data_buffer somado a out (no negedge seguinte)
3. Fase de reset: clear = 0 → data_buffer e out são zerados imediatamente

3.2.4. FSM de Controle

A máquina de estados finitos foi implementada no módulo FSM.v e é o núcleo de controle de todo o sistema. Ela gerencia a sequência de leituras, acumulações e escritas, seguindo rigorosamente os requisitos temporais da memória SRAM.

A FSM possui 13 estados definidos como parâmetros locais:

- ST_RESET_CLEAR: estado inicial, libera o sinal clear do acumulador
- ST_SET_READ_ADDR: configura o endereço de leitura atual
- ST_RDEN_ON: ativa o sinal rden para iniciar a leitura da memória
- ST_WAIT_1, ST_WAIT_2, ST_WAIT_3: estados de espera para a memória disponibilizar o dado lido
- ST_CAPTURE: ativa load para capturar o dado no acumulador e desativa rden
- ST_ADD: ativa transf para somar o dado ao acumulador
- ST_WRITE_PREP: prepara o endereço de escrita (sétimo endereço de cada bloco)
- ST_WREN_ON: ativa wren para iniciar a escrita na memória
- ST_WREN_OFF: desativa wren após um ciclo
- ST_NEXT_BLOCK: prepara transição para o próximo bloco ou finalização
- ST_READY_PULSE: ativa o sinal ready por um ciclo e reinicia a sequência

A FSM mantém dois contadores internos: block_base (0, 8, 16, 24) que indica a base de cada bloco de endereços, e index_in_block (0 a 6) que conta as leituras dentro de cada bloco. A cada ciclo completo de 7 leituras, o sistema escreve o resultado acumulado no endereço block_base + 7.

A implementação utiliza reset assíncrono para garantir inicialização consistente. Todos os sinais de controle são pulsos de duração de um ciclo de clock, sincronizados com a borda de subida. A transição entre estados respeita os requisitos da memória: após ativar rden, aguarda-se três ciclos antes

de capturar o dado; após escrita, aguarda-se a desativação de wren antes de mudar endereços. Como pode ser observado na Figura 15.

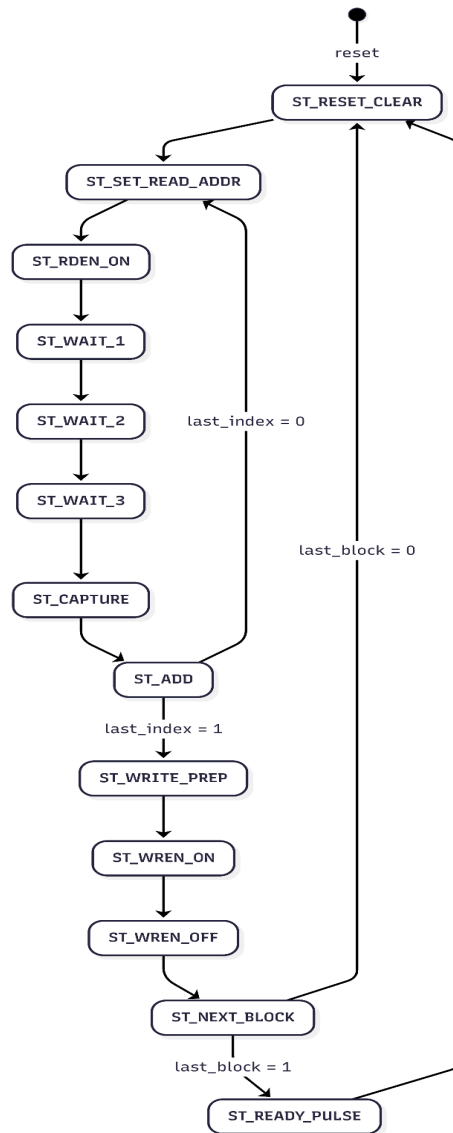


Figura 15 - Diagrama de Estado

3.2.5. Módulo TOP

O módulo TOP, implementado no arquivo TOP.v, serve como encapsulamento e integração dos módulos Acumulador e FSM. Sua função principal é isolar a interface externa dos sinais internos de controle, proporcionando uma interface limpa e padronizada para conexão com a memória e o controlador externo.

As portas do módulo TOP são:

- Entradas: clk, reset, datain[15:0] (dado lido da memória)
- Saídas: dataout[15:0] (dado a ser escrito na memória), address[4:0], rden, wren, ready

- Internamente, o TOP realiza as conexões apropriadas:
- O sinal datain é conectado diretamente à entrada in do Acumulador
- O sinal dataout é conectado à saída out do Acumulador
- Os sinais capture_data (load), clear_acc_n (clear) e add_enable (transf) da FSM são conectados aos respectivos sinais do Acumulador
- Os sinais address, rden, wren e ready da FSM são expostos como saídas do módulo TOP

Esta arquitetura permite que o sistema completo seja tratado como uma caixa preta: o controlador externo apenas fornece clock e reset, e monitora o sinal ready para saber quando os processamentos foram concluídos. Todas as complexidades de temporização e sequenciamento estão encapsuladas nos módulos internos.

3.2.6. Memória

A memória externa foi implementada utilizando o assistente de IP (Megafunction Wizard) do Quartus Prime, gerando o arquivo memoria.vhd. A configuração utilizada foi:

- Tipo: RAM de porta única (Single Port RAM)
- Largura de dados: 16 bits
- Profundidade: 32 palavras (5 bits de endereço)
- Dispositivo alvo: Cyclone IV E (família da placa DE2-115)
- Arquivo de inicialização: RAM.mif com valores pré-definidos para teste

O arquivo RAM.mif que é mostrado no Figura 16 contém valores numéricos diversos distribuídos pelos 32 endereços, permitindo verificar corretamente as operações de soma. A memória opera com clock síncrono: as leituras são registradas na saída q um ciclo após a ativação de rden, e as escritas ocorrem na borda de clock quando wren está ativo.

A interface da memória é compatível com os sinais gerados pela FSM:

- address: endereço de 5 bit
- clock: sinal de clock
- data: dado de entrada para escrita (16 bits)
- rden: enable de leitura
- wren: enable de escrita
- q: dado de saída da leitura (16 bits)

Depois da criação da memória do Quartus, foram observados sinais reservados do sistema JTAG (como altera_reserved_tck), inseridos automaticamente pelo Quartus para habilitar capacidades de debug. Tais sinais não interferem na lógica implementada e são gerenciados internamente pela ferramenta de desenvolvimento.

```

-- Quartus Prime generated Memory Initialization File (.mif)

WIDTH=16;
DEPTH=32;

ADDRESS_RADIX=UNS;
DATA_RADIX=UNS;

CONTENT BEGIN
  0 : 4821;
  1 : 15342;
  2 : 987;
  3 : 27461;
  4 : 6210;
  5 : 44123;
  6 : 19384;
  7 : 0;
  8 : 728;
  9 : 32015;
 10 : 1456;
 11 : 50291;
 12 : 8734;
 13 : 291;
 14 : 36612;
 15 : 0;
 16 : 15432;
 17 : 612;
 18 : 48976;
 19 : 7812;
 20 : 245;
 21 : 32760;
 22 : 1045;
 23 : 0;
 24 : 21987;
 25 : 389;
 26 : 47012;
 27 : 9231;
 28 : 158;
 29 : 51234;
 30 : 6401;
 31 : 0;
END;

```

Valores que vai para memória.

Figura 16 - Dados da Memória

3.2.7. Time Constraints

Para garantir o funcionamento correto na placa DE2-115, foram aplicadas restrições temporais utilizando o TimeQuest Timing Analyzer do Quartus Prime. Inicialmente, o projeto foi configurado com as definições padrão, que assumem um clock de 1 GHz para análise. Como a placa opera com um clock fixo de 50 MHz, foi necessário ajustar essas configurações.

As restrições aplicadas foram:

- create_clock: definição do clock principal com período de 20 ns (50 MHz)
- set_input_delay e set_output_delay: definição de atrasos de propagação de 2 ns (10% do período do clock) para as entradas e saídas

As análises de temporização realizadas no TimeQuest Timing Analyzer confirmaram que todas as margens de setup e hold time foram satisfeitas para a frequência de operação de 50 MHz da placa DE2-115. O relatório de temporização indicou uma frequência máxima

de operação (Fmax) compatível com as capacidades do FPGA Cyclone IV, garantindo uma margem de segurança adequada para operação estável. As verificações foram realizadas nos modelos Slow 1200 mV 85°C, Slow 1200 mV 0°C e Fast 1200 mV 0°C, abrangendo condições extremas de temperatura e tensão.

3.2.8. Simulação (Testbench)

A validação do sistema foi realizada por meio de simulações no ambiente ModelSim-Altera, integradas diretamente ao Quartus Prime. Para facilitar o processo de verificação e tornar a análise dos resultados mais organizada, foi adotada uma estratégia que permite a seleção dinâmica do testbench e a configuração automática das formas de onda durante a abertura da simulação.

No menu de configurações de simulação do Quartus (*Assignments* → *Settings* → *EDA Tool Settings* → *Simulation*), foram cadastrados múltiplos testbenches correspondentes aos diferentes níveis do projeto, a saber: tb_Acumulador, FSM_TB e TOP_TB. Dessa forma, é possível selecionar, por meio da opção *Compile test bench*, qual testbench será utilizado na simulação, sem a necessidade de alterações manuais nos arquivos ou no ambiente do ModelSim como na figura 17.

Além disso, foi utilizado um script de configuração de ondas (wave.do), associado automaticamente à simulação por meio da opção *Use script to set up simulation*. Esse script é responsável por organizar previamente os sinais de interesse em grupos, definir escalas de tempo, raízes e posicionamento das formas de onda, além de executar automaticamente o comando de simulação (*run*). Além disso, foi desenvolvido um único script de simulação que contempla os três testbenches do projeto. De acordo com o testbench selecionado, o script executa automaticamente apenas a configuração e a simulação correspondentes ao módulo escolhido.

Com essa configuração, ao iniciar o ModelSim diretamente a partir do Quartus, o ambiente de simulação é aberto já com:

- O testbench selecionado compilado;
- As formas de onda organizadas de acordo com o módulo em teste;
- A simulação executada automaticamente, permitindo a visualização imediata do comportamento do sistema.

Essa abordagem proporcionou maior eficiência no processo de verificação, reduzindo erros operacionais, facilitando a comparação entre diferentes níveis de teste e garantindo os resultados apresentados. A Figura 18 ilustra a configuração adotada no Quartus para seleção do testbench e associação do script de ondas.

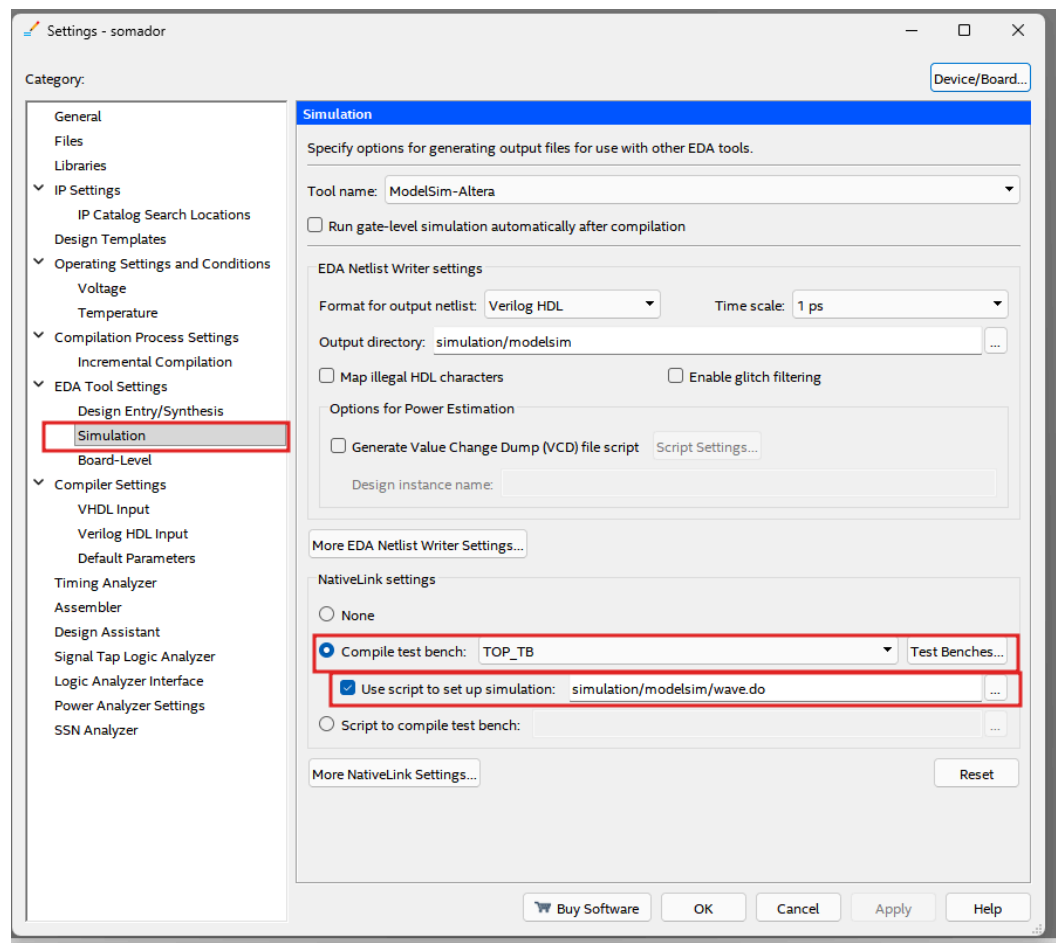


Figura 17 - Configuração do ambiente de simulação no Quartus Prime com associação do script *wave.do*

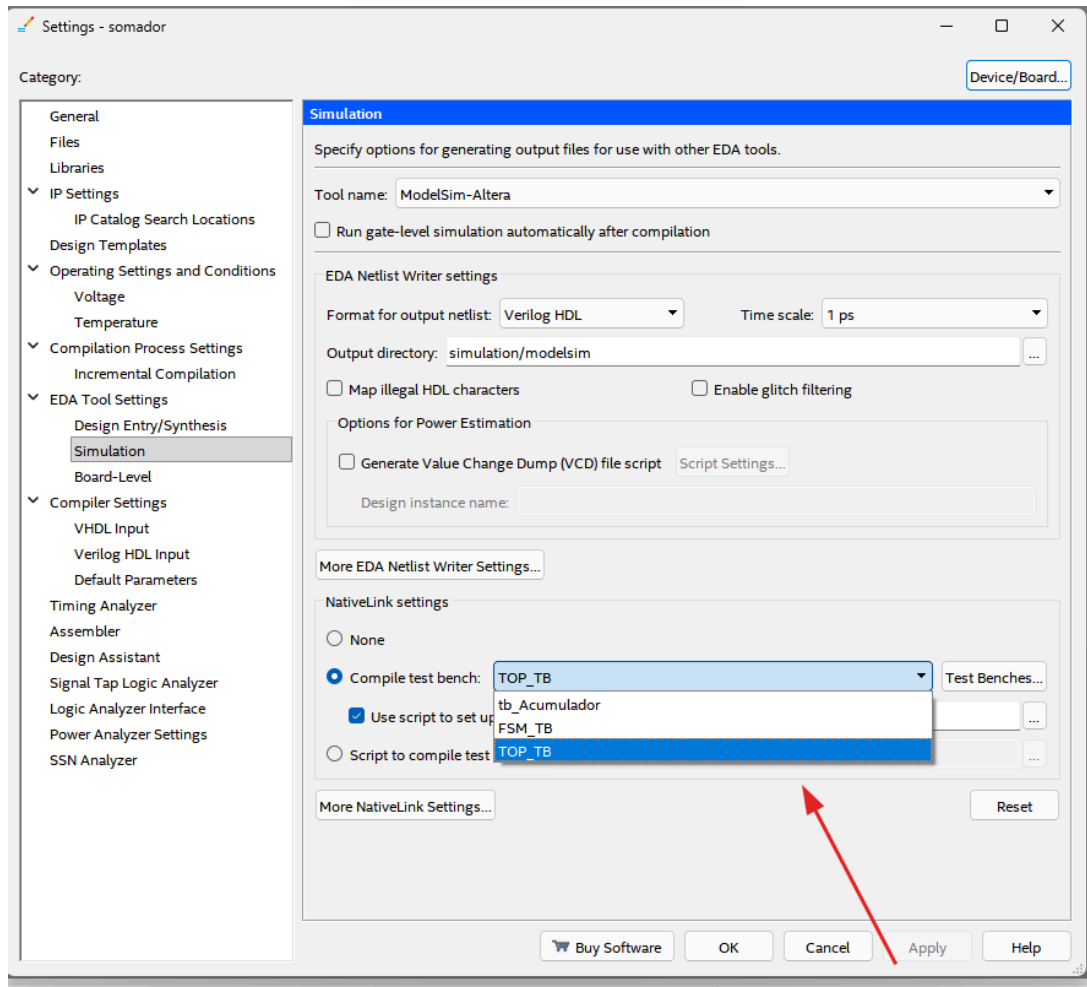


Figura 18 - Seleção dinâmica do testbench no Quartus Prime para execução da simulação no ModelSim
Esses foram os três testbenches desenvolvidos para validar cada componente do sistema:

3.2.8.1. Testbench do Acumulador (tb_Acumulador.v)

O testbench verifica a funcionalidade básica do módulo Acumulador através de uma sequência de estímulos controlada. A simulação testa

- Captura de dados com o sinal load;
- Acumulação com o sinal transf;
- Reset do acumulador com clear;
- Sequências de múltiplas operações de soma;

O testbench tb_Acumulador.v valida o módulo através de uma sequência progressiva e sincronizada com a negedge do clock. Inicia com reset ativo (clear=0) e transições controladas para testar operações básicas: primeiro, captura e soma do valor 10; em seguida, acumulação sequencial com 5 e 20, verificando resultado 35; depois, ativação de clear durante operação para testar reset imediato; finalmente, operação pós-reset com valor 7. Cada etapa emprega \$display para registrar entradas, ações e resultados esperados, enquanto \$monitor rastreia em tempo real todos os sinais e

mostra todas as etapas no console, como a mensagem ficou muito grande colocamos uma parte dele para mostrar funcionando como na imagem 20. A sincronização precisa com a borda de descida garante alinhamento temporal com a implementação do acumulador, confirmando não apenas resultados corretos (10, 15, 35, 0, 7) mas também compliance com requisitos de temporização e comportamento determinístico em todas as condições testadas. Como pode ser observado nas Figuras 19,20.



Figura 19 - Testbench do acumulador

```

# T=125000 | clk=1 clear=1 load=0 transf=1 in= 20 out= 15
# T=130000 | RESULTADO ESPERADO: out = 35 | out atual = 15
# T=130000 | clk=0 clear=1 load=0 transf=0 in= 20 out= 35
# T=135000 | clk=1 clear=1 load=0 transf=0 in= 20 out= 35
# T=140000 | CLEAR ATIVO: zerando acumulador
# T=140000 | clk=0 clear=0 load=0 transf=0 in= 20 out= 0
# T=145000 | clk=1 clear=0 load=0 transf=0 in= 20 out= 0
# T=150000 | CLEAR LIBERADO | out = 0 (esperado 0)
# T=150000 | clk=0 clear=1 load=0 transf=0 in= 20 out= 0
# T=155000 | clk=1 clear=1 load=0 transf=0 in= 20 out= 0
# T=160000 | LOAD: capturando valor 7
# T=160000 | clk=0 clear=1 load=1 transf=0 in= 7 out= 0
# T=165000 | clk=1 clear=1 load=1 transf=0 in= 7 out= 0
# T=170000 | clk=0 clear=1 load=0 transf=0 in= 7 out= 0
# T=175000 | clk=1 clear=1 load=0 transf=0 in= 7 out= 0
# T=180000 | TRANSF: acumulando valor
# T=180000 | clk=0 clear=1 load=0 transf=1 in= 7 out= 0
# T=185000 | clk=1 clear=1 load=0 transf=1 in= 7 out= 0
# T=190000 | RESULTADO FINAL ESPERADO: out = 7 | out atual = 0
# T=190000 | clk=0 clear=1 load=0 transf=0 in= 7 out= 7
# T=195000 | clk=1 clear=1 load=0 transf=0 in= 7 out= 7
# T=200000 | clk=0 clear=1 load=0 transf=0 in= 7 out= 7
# T=205000 | clk=1 clear=1 load=0 transf=0 in= 7 out= 7
# -----
# FIM DA SIMULACAO
# -----
# ** Note: $stop : C:/ENGG56/Somador/tb_Acumulador.v(156)
# Time: 210 ns Iteration: 0 Instance: /tb_Acumulador
# Break in Module tb_Acumulador at C:/ENGG56/Somador/tb_Acumulador.v line 156

```

Figura 20 - Mensagens de console resultantes da execução do testbench do acumulador

3.2.8.2. Testbench da FSM (tb_FSM.v)

O testbench da FSM (FSM_TB.v) implementa uma estratégia de validação focada em verificações temporais rigorosas e monitoramento de estado, garantindo que a máquina de estados respeite os requisitos de temporização da memória SRAM e do acumulador. Operando com clock de 50 MHz (período de 20 ns), o testbench inicia com reset ativo e, após estabilização, habilita um sistema de checks que monitora todas as transições de sinais críticos.

Funções auxiliares (base8 e mod8) decompõem o endereço em bloco e índice para log detalhado. O coração da verificação está em múltiplos processos always que rastreiam os tempos exatos de ativação/desativação de rden, wren, load e transf, calculando intervalos entre eventos e abortando a simulação caso sejam violados limites mínimos (ex.: mudança de endereço muito próxima a rden, ou load e transf ativos simultaneamente).

O testbench também verifica o fluxo completo da FSM: ao detectar address == 31, marca um endpoint e aguarda o retorno a address == 0, quando então finaliza com sucesso. Cada transição é logada no console com timestamp, endereço e estado dos sinais, proporcionando visibilidade completa do comportamento sequencial e validando não apenas a lógica funcional, mas também a conformidade temporal essencial para operação confiável com memória externa. Na Figura 21 é mostrado um trecho da waveform da execução e dos sinais da FSM. A execução completa pode ser observada na Figura 22, enquanto as mensagens de console são apresentadas na Figura 23.

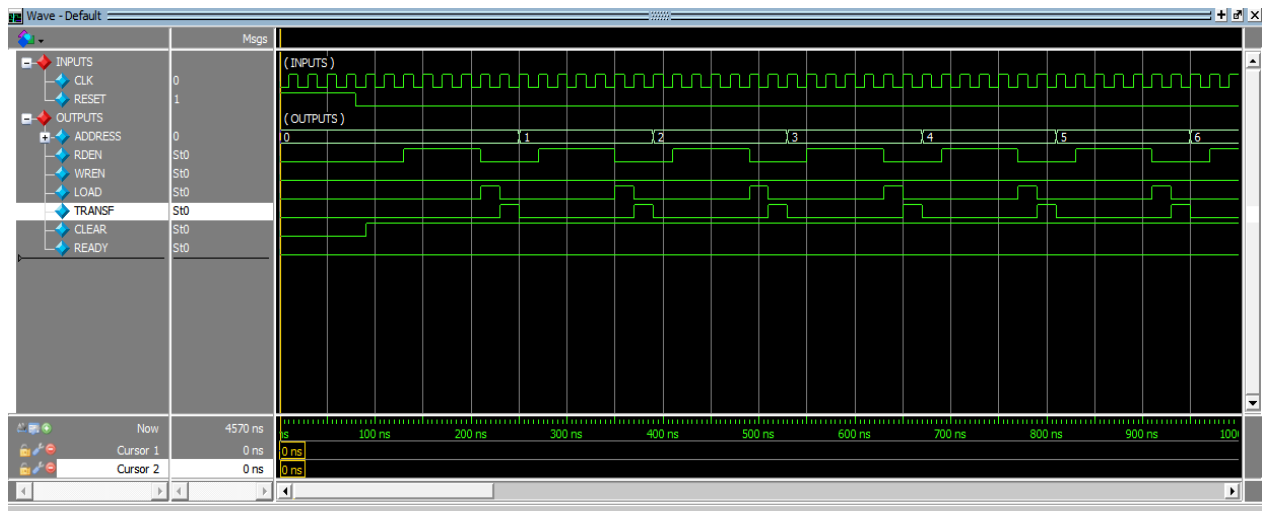


Figura 21 - Trecho da execução do testbench da FSM

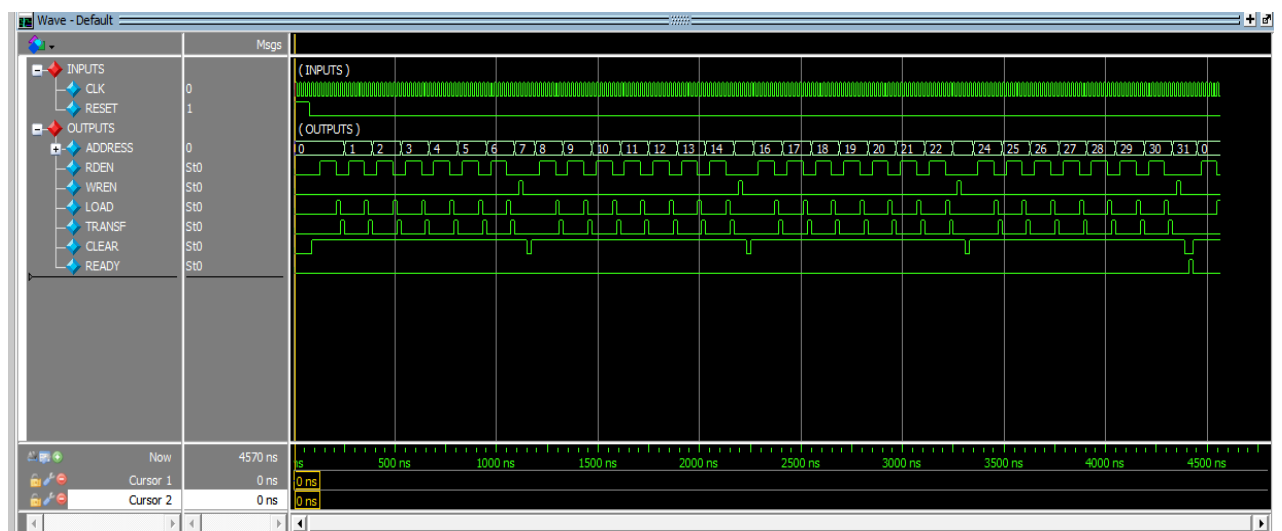


Figura 22 - Testbench completo da FSM

```

# T=4330 | clkâ | addr=30 (base=24 idx=6) | rden=0 wren=0 load=0 transf=1 clear=1 ready=0 | reset=0
# T=4330 | ADDRESS mudou -> 31 (base=24 idx=7)
# T=4330 | Endpoint atingido (address==31).
# T=4330 | TRANSF=0 (fim do pulso)
# T=4350 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4350 | WREN=1 (escrita iniciada) addr=31
# T=4370 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=1 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4370 | WREN=0 (escrita encerrada) addr=31
# T=4390 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4390 | CLEAR=0 (zera/limpa acumulador entre blocos)
# T=4410 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=0 load=0 transf=0 clear=0 ready=0 | reset=0
# T=4410 | READY=1 (fim da sequencia completa)
# T=4430 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=0 load=0 transf=0 clear=0 ready=1 | reset=0
# T=4430 | CLEAR=1 (acumulador liberado para operar)
# T=4450 | clkâ | addr=31 (base=24 idx=7) | rden=0 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4450 | ADDRESS mudou -> 0 (base=0 idx=0)
# T=4470 | clkâ | addr=0 (base=0 idx=0) | rden=0 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4470 | RDEN=1 (leitura iniciada) addr=0
# T=4490 | clkâ | addr=0 (base=0 idx=0) | rden=1 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4510 | clkâ | addr=0 (base=0 idx=0) | rden=1 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4530 | clkâ | addr=0 (base=0 idx=0) | rden=1 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4550 | clkâ | addr=0 (base=0 idx=0) | rden=1 wren=0 load=0 transf=0 clear=1 ready=0 | reset=0
# T=4550 | RDEN=0 (leitura encerrada) addr=0
# T=4550 | LOAD=1 (captura dado) addr=0
# T=4570 | Simulacao completada com sucesso. (voltou para address=0)
# ** Note: $stop : C:/ENGG56/Somador/tb_FSM.v(118)
# Time: 4570 ns Iteration: 0 Instance: /FSM_TB
# Break in Module FSM_TB at C:/ENGG56/Somador/tb_FSM.v line 118
VSIM 7>]

```

Figura 23 - Mensagens de console resultantes da execução do testbench do FSM

3.2.8.3. Testbench do Sistema Completo (tb_TOP.v)

O testbench integra todos os componentes: TOP, Acumulador, FSM e a memória real (instanciada a partir do arquivo memoria.vhd). A estratégia de teste inclui:

- Fase de clonagem: leitura de todos os endereços da memória antes da execução para criar um espelho dos dados iniciais
- Execução completa do sistema processando os quatro blocos
- Monitoramento das escritas nos endereços de resultado (7, 15, 23, 31)
- Verificação do sinal ready ao final do processamento

O testbench também inclui um mecanismo de timeout que interrompe a simulação caso ela entre em loop infinito. Durante a execução, são exibidos logs detalhados de todas as operações de leitura e escrita. Ao final da simulação, foi confirmado que os resultados escritos na memória correspondem às somas esperadas dos valores iniciais, validando o funcionamento completo do sistema. Esse testbench pode ser visualizado nas Figuras 24,25,26,27 e 28.

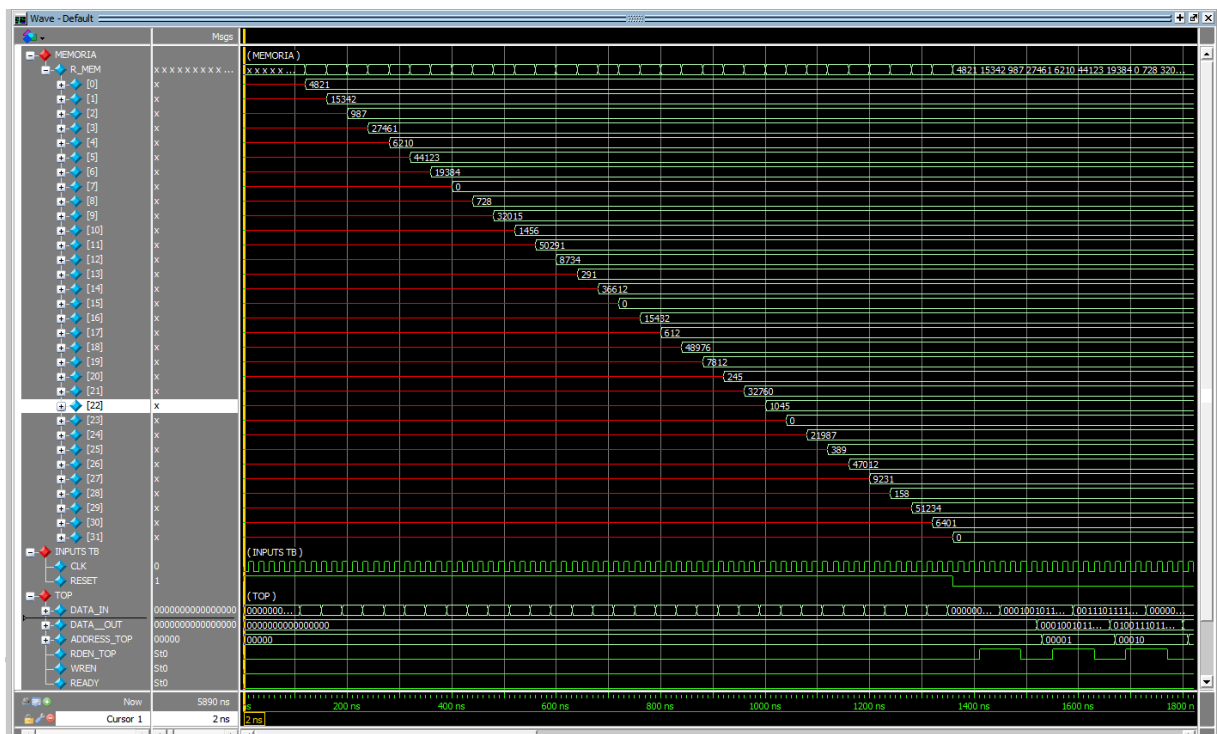


Figura 24 - Clonagem de memória RAM no Testbench do TOP

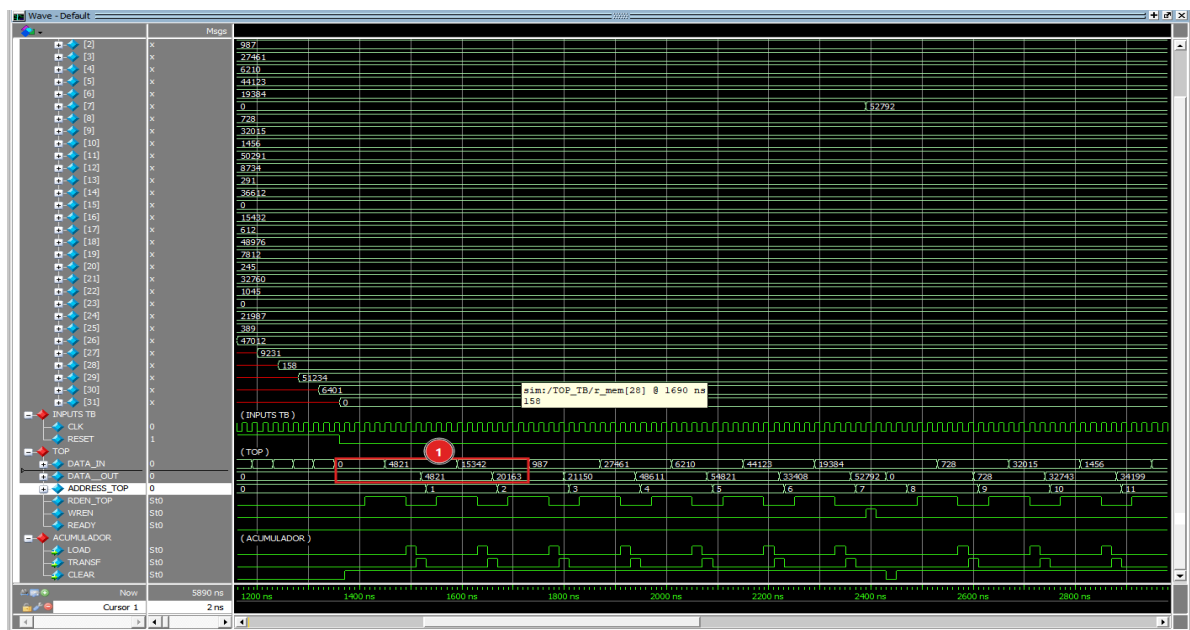


Figura 25 - Execução do testbench do módulo TOP com um exemplo do acumulador

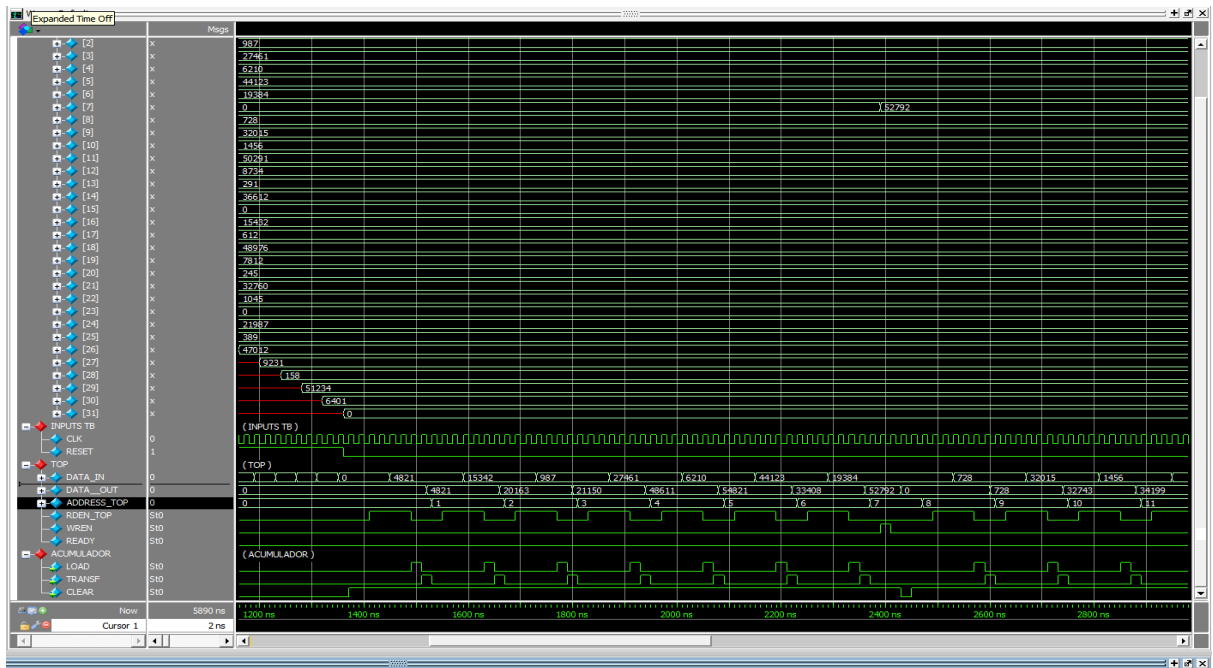


Figura 26 - Execução do testbench do módulo TOP do Somador

```

Transcript
sim:/IOP_TB/top_teste/acc/load \
sim:/TOP_TB/top_teste/acc/transf \
sim:/TOP_TB/top_teste/acc/clear
VSIM 12> run -all

# =====
# INICIO DA SIMULACAO - TOP
# Período=20ns | MeioPeríodo=10ns
# =====
# T=80 | [CLONE] TOP pausado (clk_top=0). Clonando RA
# T=120 | [CLONE] RAM[0] = 0x12d5 (4821)
# T=160 | [CLONE] RAM[1] = 0x3bee (15342)
# T=200 | [CLONE] RAM[2] = 0x03db (987)
# T=240 | [CLONE] RAM[3] = 0x6b45 (27461)
# T=280 | [CLONE] RAM[4] = 0x1842 (6210)
# T=320 | [CLONE] RAM[5] = 0xac5b (44123)
# T=360 | [CLONE] RAM[6] = 0x4bb8 (19384)
# T=400 | [CLONE] RAM[7] = 0x0000 (0)
# T=440 | [CLONE] RAM[8] = 0x02d8 (728)
# T=480 | [CLONE] RAM[9] = 0x7d0f (32015)
# T=520 | [CLONE] RAM[10] = 0x05b0 (1456)
# T=560 | [CLONE] RAM[11] = 0xc473 (50291)
# T=600 | [CLONE] RAM[12] = 0x221e (8734)
# T=640 | [CLONE] RAM[13] = 0x0123 (291)
# T=680 | [CLONE] RAM[14] = 0x8f04 (36612)
# T=720 | [CLONE] RAM[15] = 0x0000 (0)
# T=760 | [CLONE] RAM[16] = 0x3c48 (15432)
# T=800 | [CLONE] RAM[17] = 0x0264 (612)
# T=840 | [CLONE] RAM[18] = 0xbf50 (48976)
# T=880 | [CLONE] RAM[19] = 0x1e84 (7812)
# T=920 | [CLONE] RAM[20] = 0x00f5 (245)
# T=960 | [CLONE] RAM[21] = 0x7ff8 (32760)
# T=1000 | [CLONE] RAM[22] = 0x0415 (1045)
# T=1040 | [CLONE] RAM[23] = 0x0000 (0)
# T=1080 | [CLONE] RAM[24] = 0x55e3 (21987)
# T=1120 | [CLONE] RAM[25] = 0x0185 (389)
# T=1160 | [CLONE] RAM[26] = 0xb7a4 (47012)
# T=1200 | [CLONE] RAM[27] = 0x240f (9231)
# T=1240 | [CLONE] RAM[28] = 0x009e (158)
# T=1280 | [CLONE] RAM[29] = 0xc822 (51234)
# T=1320 | [CLONE] RAM[30] = 0x1901 (6401)
# T=1360 | [CLONE] RAM[31] = 0x0000 (0)
# T=1360 | [CLONE] Memória RAM Clonada.
# T=1360 | Reset liberado (reset=0). Iniciando TOP.

```

Figura 27 - Mensagens de console resultantes da execução do testbench do TOP

```

Transcript
# T=5130 | [TOP] addr_top=27 | rden=1 wren=0 | datain=0x240f dataout=0x0f0c | ready=0
# T=5150 | [TOP] addr_top=27 | rden=1 wren=0 | datain=0x240f dataout=0x0f0c | ready=0
# T=5170 | [TOP] addr_top=27 | rden=0 wren=0 | datain=0x240f dataout=0x0f0c | ready=0
# T=5190 | [TOP] addr_top=27 | rden=0 wren=0 | datain=0x240f dataout=0x331b | ready=0
# T=5210 | [TOP] addr_top=28 | rden=0 wren=0 | datain=0x240f dataout=0x331b | ready=0
# T=5230 | [TOP] addr_top=28 | rden=1 wren=0 | datain=0x240f dataout=0x331b | ready=0
# T=5250 | [TOP] addr_top=28 | rden=1 wren=0 | datain=0x240f dataout=0x331b | ready=0
# T=5270 | [TOP] addr_top=28 | rden=1 wren=0 | datain=0x009e dataout=0x331b | ready=0
# T=5290 | [TOP] addr_top=28 | rden=1 wren=0 | datain=0x009e dataout=0x331b | ready=0
# T=5310 | [TOP] addr_top=28 | rden=0 wren=0 | datain=0x009e dataout=0x331b | ready=0
# T=5330 | [TOP] addr_top=28 | rden=0 wren=0 | datain=0x009e dataout=0x33b9 | ready=0
# T=5350 | [TOP] addr_top=29 | rden=0 wren=0 | datain=0x009e dataout=0x33b9 | ready=0
# T=5370 | [TOP] addr_top=29 | rden=1 wren=0 | datain=0x009e dataout=0x33b9 | ready=0
# T=5390 | [TOP] addr_top=29 | rden=1 wren=0 | datain=0x009e dataout=0x33b9 | ready=0
# T=5410 | [TOP] addr_top=29 | rden=1 wren=0 | datain=0xc822 dataout=0x33b9 | ready=0
# T=5430 | [TOP] addr_top=29 | rden=1 wren=0 | datain=0xc822 dataout=0x33b9 | ready=0
# T=5450 | [TOP] addr_top=29 | rden=0 wren=0 | datain=0xc822 dataout=0x33b9 | ready=0
# T=5470 | [TOP] addr_top=29 | rden=0 wren=0 | datain=0xc822 dataout=0xfbdb | ready=0
# T=5490 | [TOP] addr_top=30 | rden=0 wren=0 | datain=0xc822 dataout=0xfbdb | ready=0
# T=5510 | [TOP] addr_top=30 | rden=1 wren=0 | datain=0xc822 dataout=0xfbdb | ready=0
# T=5530 | [TOP] addr_top=30 | rden=1 wren=0 | datain=0xc822 dataout=0xfbdb | ready=0
# T=5550 | [TOP] addr_top=30 | rden=1 wren=0 | datain=0x1901 dataout=0xfbdb | ready=0
# T=5570 | [TOP] addr_top=30 | rden=1 wren=0 | datain=0x1901 dataout=0xfbdb | ready=0
# T=5590 | [TOP] addr_top=30 | rden=0 wren=0 | datain=0x1901 dataout=0xfbdb | ready=0
# T=5610 | [TOP] addr_top=30 | rden=0 wren=0 | datain=0x1901 dataout=0x14dc | ready=0
# T=5630 | [TOP] addr_top=31 | rden=0 wren=0 | datain=0x1901 dataout=0x14dc | ready=0
# T=5630 | [WRITE] addr=31 | dataout=0x14dc (5340)
# T=5650 | [TOP] addr_top=31 | rden=0 wren=1 | datain=0x1901 dataout=0x14dc | ready=0
# T=5670 | [TOP] addr_top=31 | rden=0 wren=0 | datain=0x1901 dataout=0x14dc | ready=0
# T=5690 | [TOP] addr_top=31 | rden=0 wren=0 | datain=0x1901 dataout=0x0000 | ready=0
# T=5690 | READY=1 (fim). Simulacao OK. Parando com $stop.
# T=5710 | [TOP] addr_top=31 | rden=0 wren=0 | datain=0x1901 dataout=0x0000 | ready=1
# T=5730 | [TOP] addr_top=31 | rden=0 wren=0 | datain=0x1901 dataout=0x0000 | ready=0
# T=5750 | [TOP] addr_top=0 | rden=0 wren=0 | datain=0x1901 dataout=0x0000 | ready=0
# T=5770 | [TOP] addr_top=0 | rden=1 wren=0 | datain=0x1901 dataout=0x0000 | ready=0
# T=5790 | [TOP] addr_top=0 | rden=1 wren=0 | datain=0x1901 dataout=0x0000 | ready=0
# T=5810 | [TOP] addr_top=0 | rden=1 wren=0 | datain=0x12d5 dataout=0x0000 | ready=0
# T=5830 | [TOP] addr_top=0 | rden=1 wren=0 | datain=0x12d5 dataout=0x0000 | ready=0
# T=5850 | [TOP] addr_top=0 | rden=0 wren=0 | datain=0x12d5 dataout=0x0000 | ready=0
# T=5870 | [TOP] addr_top=0 | rden=0 wren=0 | datain=0x12d5 dataout=0x12d5 | ready=0
# ** Note: $stop : C:/ENGG56/Somador/tb_TOP.v(164)
# Time: 5890 ns Iteration: 0 Instance: /TOP_TB
# Break in Module TOP_TB at C:/ENGG56/Somador/tb_TOP.v line 164

```

Figura 28 - Mensagens de console resultantes da execução do testbench do TOP

3.3. CODIFICAÇÃO DE VÍDEO MPEG

3.3.1. Descrição do problema

No processo de codificação de vídeo MPEG, utiliza-se a Transformada Cosseno Discreta (DCT) para converter as amostras de brilho e cor dos pixels do domínio espacial para o domínio da frequência. Isso permite a compressão do vídeo pela eliminação de amplitudes baixas e de altas frequências (quantização).

O circuito implementado como na Figura 29 calcula a transformada inversa (IDCT) para blocos de 8×8 pixels, seguindo a fórmula matemática como na Figura 30:

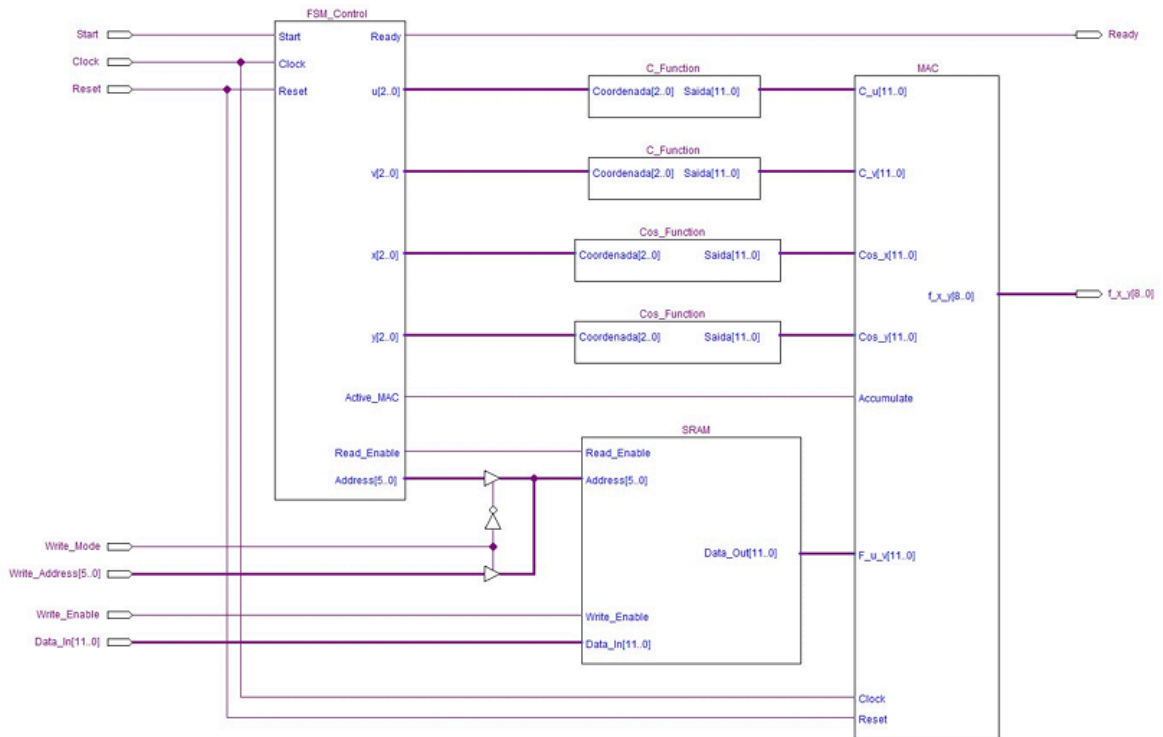


Figura 29 - Circuito do Projeto 3

$$f(x, y) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \left[C(u) \cdot C(v) \cdot F(u, v) \cdot \cos \left(\frac{(2x+1)u\pi}{2N} \right) \cdot \cos \left(\frac{(2y+1)v\pi}{2N} \right) \right]$$

Figura 30 - Equação do Projeto 3

- x,y: coordenadas espaciais (0 a 7)
- u,v: coordenadas de frequência (0 a 7)
- F(u,v): coeficientes de frequência (12 bits, armazenados em SRAM)
- C(u),C(v): funções de normalização obtidas por lookup tables

O objetivo é implementar uma máquina de estados finitos (FSM) denominada FSM_Control, responsável por gerar os sinais de controle para os módulos auxiliares (C_Function, Cos_Function, MAC) e gerenciar as leituras dos coeficientes da SRAM.

3.3.2. Arquitetura do módulo

O sistema implementado para o cálculo da IDCT segue uma arquitetura modular controlada por uma máquina de estados finitos (FSM) do tipo Moore. A unidade FSM_Control atua como núcleo de coordenação, gerando sinais de temporização para três módulos especializados: uma unidade multiplicadora-acumuladora (MAC) responsável pelas operações aritméticas, duas memórias de

consulta (Lookup Tables) que fornecem os valores pré-calculados das funções $C(u)$ e cosseno, e uma SRAM que armazena os coeficientes de frequência $F(u,v)$.

A operação segue um fluxo pipeline rigoroso: primeiro, a FSM ativa a leitura da SRAM e endereça as Lookup Tables, após um ciclo de espera para a latência da memória, envia um pulso para a unidade MAC realizar a multiplicação e acumulação. Este ciclo básico se repete 64 vezes para cada pixel (x,y) , enquanto a FSM gerencia a complexa iteração aninhada sobre as quatro variáveis x,y,u,v (cada variando de 0 a 7).

Para a implementação física na placa DE2-115, foram definidas as seguintes atribuições de pinos: o clock do sistema utiliza o oscilador de 50 MHz da placa; o sinal de reset foi conectado ao botão SW[1]; e o sinal de start ao switch SW[0]. As saídas de controle (ready, act_mac, Read_Enable) foram mapeadas para os LEDs verdes LEDG[0] a LEDG[3], enquanto as variáveis x,y,u,v (3 bits cada) foram distribuídas em grupos de três LEDs vermelhos para monitoramento visual. O barramento de endereço da SRAM (6 bits) ocupou os LEDs vermelhos de 12 a 17.

Ao final do processamento completo do bloco 8×8 , quando todas as combinações foram exploradas, a FSM ativa o sinal ready por um ciclo de clock, sinalizando a conclusão do cálculo e a disponibilidade dos valores reconstruídos (x,y) . O diagrama de blocos pode ser visualizado na Figura 31.

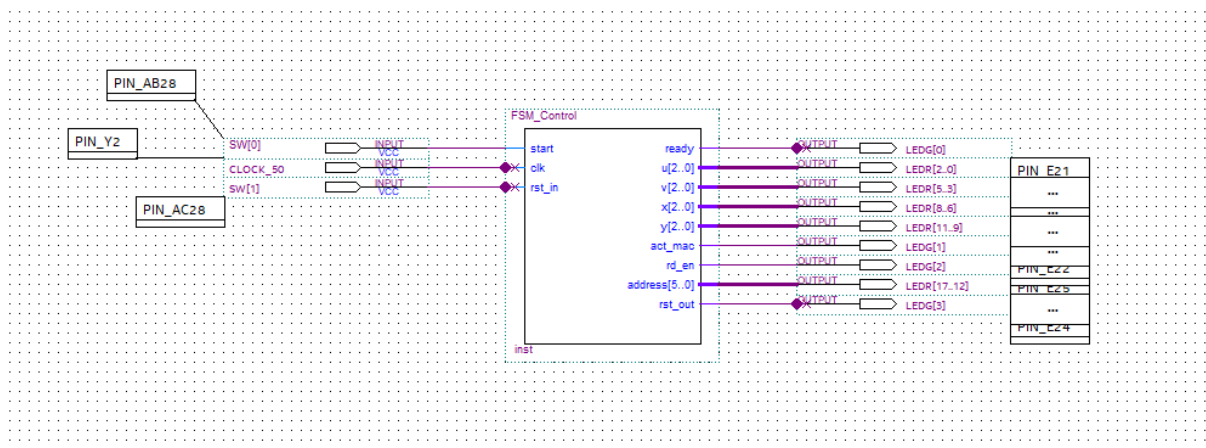


Figura 31 - Diagrama de conexões do módulo FSM Control

3.3.3. Máquina de MOORE

O controle do sistema IDCT foi implementado usando uma Máquina de Estados Finitos (FSM) do tipo Moore. Neste tipo de máquina, os sinais de saída como rd_en , act_mac e $ready$ dependem apenas do estado atual em que o sistema se encontra. Isso torna o funcionamento mais simples e previsível, pois cada estado tem suas próprias saídas fixas, que não mudam enquanto a máquina permanece nele.

O diagrama de estados da FSM_Control, mostrado na Figura 32, ilustra bem essa ideia. Cada caixa (estado) indica claramente quais sinais de controle estão ativos ali. Por exemplo, no estado RD_PULSE, o sinal rd_en fica sempre ligado; no estado ACCUM, é a vez do act_mac; e no estado DONE, o sinal ready é ativado. Essa organização facilita entender e projetar o fluxo de controle.

A máquina possui nove estados que se alternam no ritmo do clock de 50 MHz. A sequência começa com um pulso em start e passa por estágios bem definidos: inicialização (INIT), preparação do endereço (SET_ADDR), leitura da memória (RD_PULSE, WAIT_MEM), processamento (ACCUM, GAP) e atualização dos contadores (NEXT). Ao final de cada bloco de pixels, a máquina vai para o estado DONE e depois recomeça. Usar o modelo Moore ajuda a garantir que os sinais de controle tenham a duração certa e o momento certo, o que é essencial para o funcionamento correto da memória SRAM e da unidade de cálculo MAC.

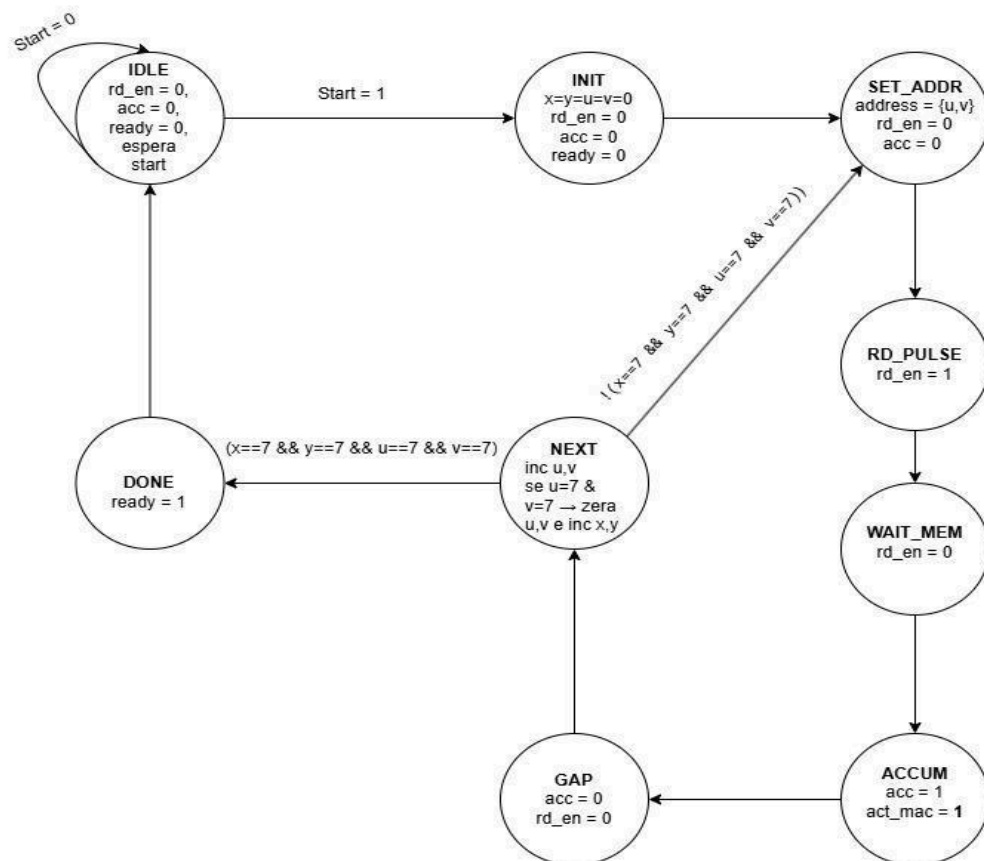


Figura 32 - Máquina de Estados Finitos (FSM) do tipo Moore do projeto 3

3.3.4. FSM de Controle

O código Verilog do módulo FSM_Control foi escrito seguindo uma estrutura clara com três partes principais: o registro do estado atual, a lógica para escolher o próximo estado e a definição das saídas.

O sistema começa com um reset assíncrono que coloca a máquina no estado IDLE. A cada borda de subida do clock, a FSM muda para o próximo estado calculado. A transição entre estados

segue um fluxo fixo: depois de receber o sinal start, passa por INIT, SET_ADDR, RD_PULSE, WAIT_MEM, ACCUM, GAP e NEXT, voltando para SET_ADDR ou indo para DONE.

A parte mais importante do código é a lógica de contagem que controla as variáveis x, y, u, v. No estado NEXT, primeiro tenta aumentar v. Se v já estiver no máximo (7), zera v e aumenta u. Quando tanto u quanto v chegam a 7, significa que todos os coeficientes para o pixel atual foram processados. Então, zera u e v e passa para o próximo pixel: aumenta x, e se x já for 7, zera x e aumenta y. Como pode ser visto no código abaixo.

```
// Exemplo do incremento no estado NEXT:
if (!(u==3'd7 && v==3'd7)) begin
    if (v < 3'd7) v <= v + 1;
    else begin
        v <= 3'd0;
        u <= u + 1;
    end
end else begin
    u <= 3'd0; v <= 3'd0;
    // incrementa x, y...
end
```

Os sinais de saída são gerados de forma simples e direta: rd_en fica ativo apenas no estado RD_PULSE, act_mac só no estado ACCUM e ready apenas no estado DONE. O endereço para a memória SRAM é montado juntando os bits de u e v ({u,v}) e é atualizado no estado SET_ADDR.

Quando todas as variáveis (x, y, u, v) atingem o valor 7 ao mesmo tempo, a condição done_all fica verdadeira e a FSM vai para o estado DONE, ativando o sinal ready por um ciclo de clock para avisar que o processamento do bloco de 64 pixels foi concluído.

3.3.5. Time Constraints

Para garantir o funcionamento correto na placa DE2-115, foram aplicadas restrições temporais no projeto. O Quartus foi configurado para operar com um clock de 50 MHz (período de 20 ns), usando o comando create_clock. Também foram definidos atrasos de entrada e saída de 2 ns para todos os sinais, garantindo margem suficiente para os tempos de setup e hold.

Após aplicar essas configurações, o Timing Analyzer foi executado novamente. A análise confirmou que todas as temporizações estão dentro dos limites necessários para operação estável a 50 MHz, com uma frequência máxima alcançável (Fmax) que oferece boa margem de segurança.

Verificações foram feitas nos modelos críticos Slow 1200 mV 85°C e Fast 1200 mV 0°C, validando o funcionamento em condições extremas de temperatura e tensão.

3.3.6. Simulação (Testbench)

A validação do módulo FSM_Control foi realizada através de um testbench autoverificável que monitora automaticamente o comportamento do circuito. Após aplicar um pulso no sinal start, o testbench executa seis verificações críticas em tempo real:

- Pulso de leitura - rd_en deve ter duração de apenas 1 ciclo;
- Timing do MAC - act_mac deve ocorrer 2 ciclos após rd_en;
- Endereço correto - address deve ser igual a {u,v} durante leitura;
- Sequência de contagem - Verifica ordem correta: $v \rightarrow u \rightarrow x \rightarrow y$;
- Mudança de pixel - Quando $u=v=7$, deve incrementar x (ou se $x=7$, zera x e incrementa y);
- Sinal de pronto - ready só ativa após $x=y=u=v=7$.

O testbench executa automaticamente todas as 4096 combinações de índices (8 valores para cada variável x, y, u, v). Cada verificação é feita em tempo real: se alguma falhar, a simulação é interrompida com uma mensagem de erro específica. Caso todas passem, o console do ModelSim exibe a confirmação "Simulação finalizou com sucesso", comprovando que a FSM funciona corretamente tanto na lógica quanto no timing. Todas as operações são registradas em um arquivo de log (fsm_log.txt) para análise posterior. As Figuras 33,34,35,36 e 37 apresentam os resultados visuais da simulação do testbench do módulo FSM_Control. Elas demonstram desde o detalhamento dos ciclos de leitura e cálculo até a confirmação final de sucesso no console. Cada figura captura um aspecto essencial do comportamento validado do circuito.

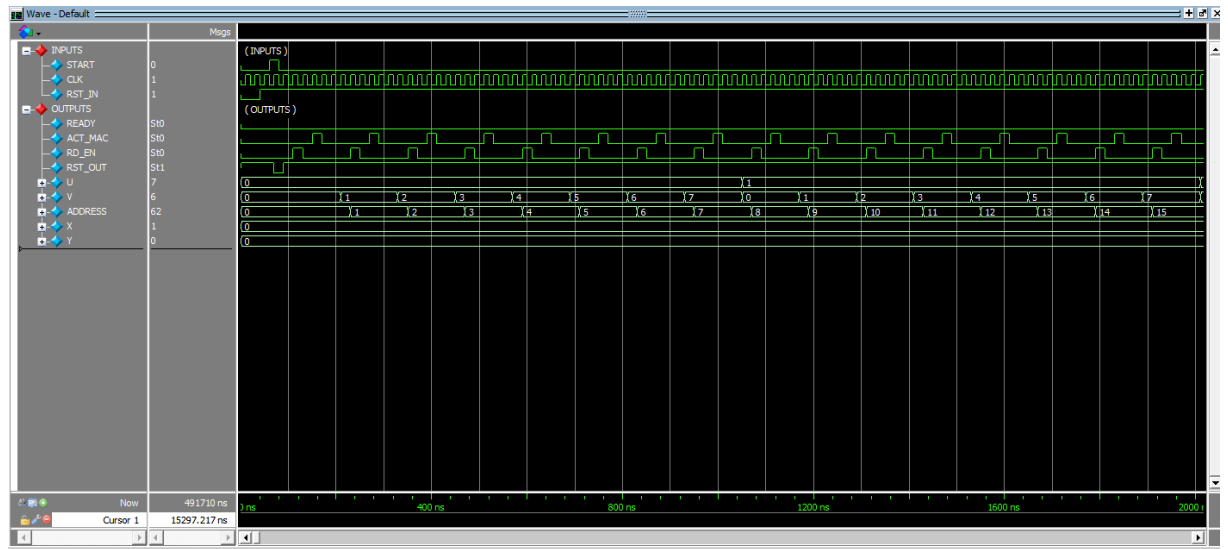


Figura 33 - Ciclo de Leitura Detalhe da variação de u, v e address durante leitura da SRAM

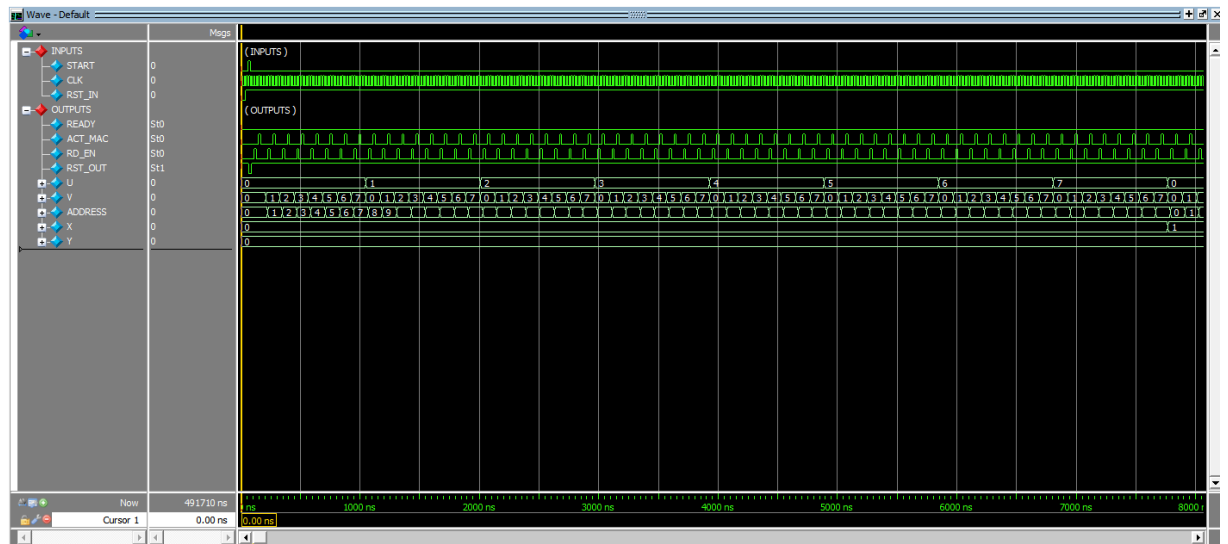


Figura 34 - Cálculo de Pixel Ciclo completo do cálculo de $f(x,y)$ desde rd_en até act_mac.

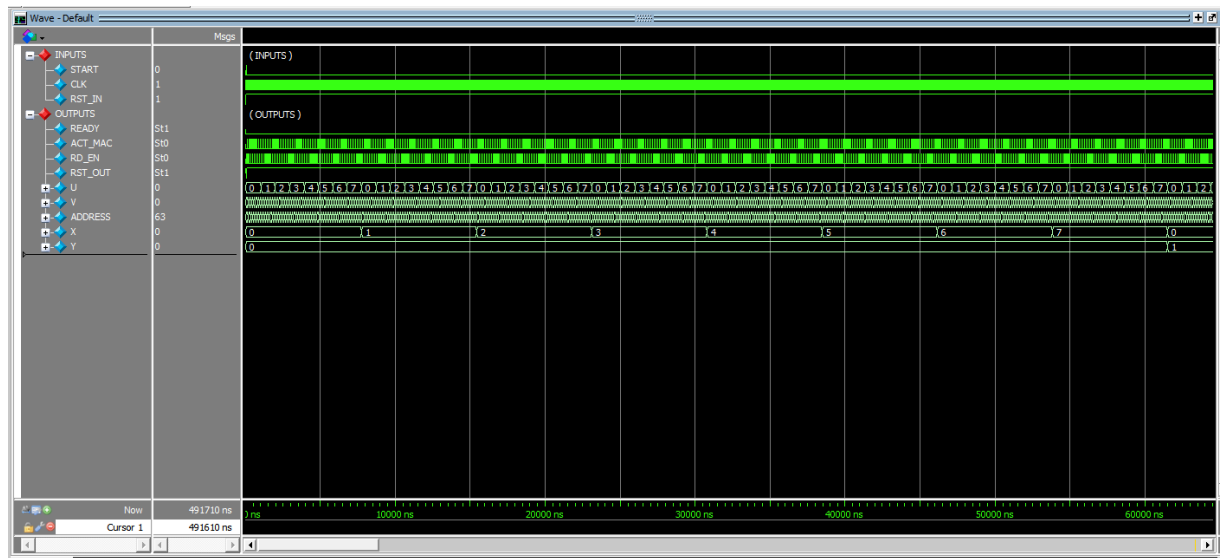


Figura 35 - Variação de X Progressão completa da coordenada x de 0 a 7.

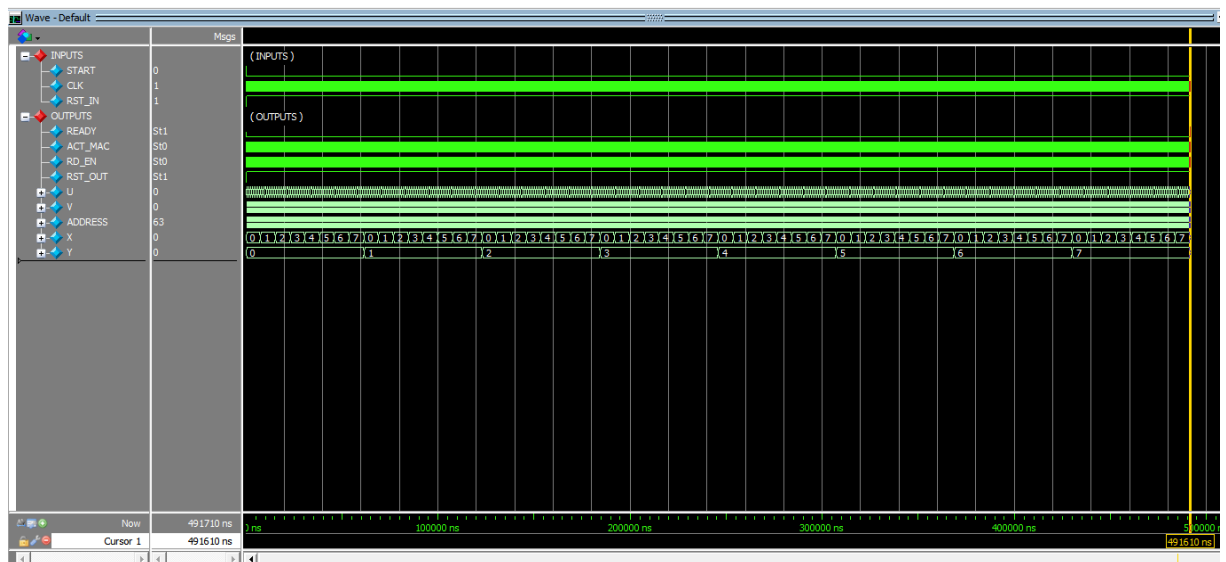


Figura 36 - Variação de Y Progressão completa da coordenada y de 0 a 7.

```
#
# run -all
# Simulacao finalizou com sucesso
# ** Error (suppressible): (vsim-PLI-3085) $fwrite : Argument 1 is an unknown file descriptor.
# Time: 491630 ns Iteration: 1 Process: /FSM_Control_TB/#ALWAYS#91 File: C:/ENGG56/MPEG/tb_FSM_Control.v Line: 96
# ** Note: $stop : C:/ENGG56/MPEG/tb_FSM_Control.v(222)
# Time: 491710 ns Iteration: 0 Instance: /FSM_Control_TB
# Break in Module FSM_Control_TB at C:/ENGG56/MPEG/tb_FSM_Control.v line 222

/SIM 2>
```

Figura 37 - Mensagens no console do testbench de FSM_Control

3.4. SINALEIRA

3.4.1. Descrição do problema

O desafio apresentado na atividade pede o desenvolvimento de um circuito otimizado (a partir de um mapa de Karnaugh) para o problema de gestão de tráfego apresentado. Neste, quatro sensores digitais (A, B, C e D) detectam a presença de veículos em um cruzamento entre uma via principal (sentido leste-oeste, sensores C e D) e uma via secundária (sentido norte-sul, sensores A e B).

Com base nas combinações de sinais fornecidas por esses sensores, o circuito deve decidir qual semáforo permanecerá verde, seguindo as definições de projeto definidas pela atividade. O projeto deve gerar duas saídas, **LO** e **NS**, correspondentes aos sinais leste-oeste e norte-sul, garantindo que nunca ambos estejam verdes ou vermelhos simultaneamente. A solução final deve ser implementada em Verilog, utilizando expressões lógicas minimizadas.

3.4.2. Simplificação lógica

A partir das restrições fornecidas pela questão, foi montada a tabela-verdade (tabela 1) e, a partir desta, foram construídos os mapas de Karnaugh (tabela 2 e tabela 3) referentes aos semáforos do cruzamento descrito.

Utilizando como referências os mapas desenvolvidos, é possível chegar nas equações booleanas otimizadas para as saídas. Essas equações serão utilizadas no módulo verilog para atribuição dos valores de saída dos semáforos.

$$NS = (A \text{ or } B) \text{ and } \sim C \text{ and } \sim D$$

$$LO = \sim NS$$

A 	B	C	D	L_O	N_S
0	0	0	0	1	0
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	1	0

Tabela 1: tabela-verdade do cruzamento.

N_S				
AB/CD	00	01	10	11
00	0	0	0	0
01	1	0	0	0
10	1	0	0	0
11	1	0	0	0

Tabela 2: mapa de Karnaugh da saída N_S.

L_O				
AB/CD	00	01	10	11
00	1	1	1	1
01	0	1	1	1
10	0	1	1	1
11	0	1	1	1

Tabela 3: mapa de Karnaugh da saída L_O.

3.4.3. Implementação em Verilog

A implementação em verilog é relativamente simples, sendo necessário apenas um módulo cujos outputs (LO e NS) são feitos a partir de assigns de acordo com as equações booleanas simplificadas descritas na seção anterior.

```
module sinaleira (A, B, C, D, NS, LO);  
    input  A, B, C, D;  
    output NS;  
    output LO;  
    assign NS = (A | B) & (~C) & (~D);  
    assign LO = ~NS;  
endmodule
```

3.4.4. Time Constraints e Pin Assignments

Embora o circuito seja puramente combinacional e não possua um sinal de clock físico, foi realizada uma análise de timing utilizando o TimeQuest Timing Analyzer do Quartus Prime. Para isso, definiu-se um clock virtual com período de 20 ns, correspondente a uma frequência de 50 MHz, compatível com a operação geral da placa DE2-115.

Com o clock virtual como referência, foram estabelecidos atrasos de entrada e saída de 2 ns (set_input_delay e set_output_delay), garantindo que os atrasos de propagação internos do circuito combinacional respeitem a janela temporal equivalente a um ciclo do clock do sistema. Essa abordagem assegura que o sinal se estabilize dentro do tempo esperado, mesmo em condições variáveis de temperatura e tensão.

No diagrama de conexões da figura abaixo, é possível localizar as entradas (A, B, C, D) e saídas (NS, LO) de um único bit, como definidas na tabela verdade, sendo representados por 4 *switchs* e 2 LED 's respectivamente.

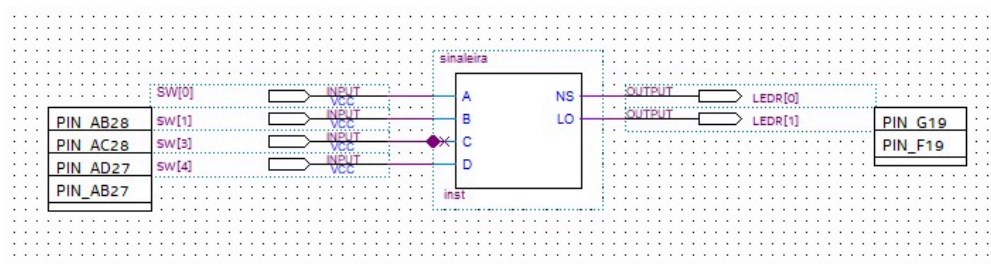


Figura 38 - Diagrama de conexões do módulo

3.4.5. Simulação (Testbench)

Para simulação final foram verificadas todas as possíveis combinações de entradas e seus respectivos *outputs*, comparando-os com os valores definidos na tabela-verdade (tabela 1). No gráfico *waveform* da Figura 39 é possível validar as respostas do circuito, visto a coerência com a tabela-verdade. A saída do console pode ser visualizada na Figura 40, onde é possível visualizar a permutação de *inputs* e a coerência entre o *output* esperado e o obtido.

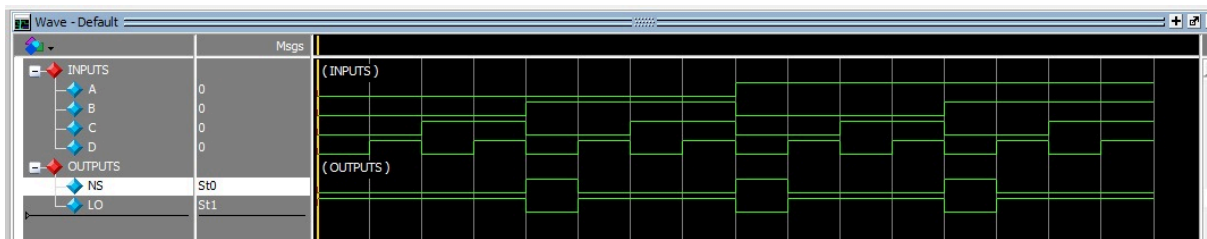


Figura 39 - Waveform do Testbench do semáforo

```
VSIM 6> run -all
#
# =====
#  CASO | A B C D | (A|B)&~C&~D | NS LO | RESULTADO
#  =====
#  0 | 0 0 0 0 | 0 | 0 1 | OK
#  1 | 0 0 0 1 | 0 | 0 1 | OK
#  2 | 0 0 1 0 | 0 | 0 1 | OK
#  3 | 0 0 1 1 | 0 | 0 1 | OK
#  4 | 0 1 0 0 | 1 | 1 0 | OK
#  5 | 0 1 0 1 | 0 | 0 1 | OK
#  6 | 0 1 1 0 | 0 | 0 1 | OK
#  7 | 0 1 1 1 | 0 | 0 1 | OK
#  8 | 1 0 0 0 | 1 | 1 0 | OK
#  9 | 1 0 0 1 | 0 | 0 1 | OK
# 10 | 1 0 1 0 | 0 | 0 1 | OK
# 11 | 1 0 1 1 | 0 | 0 1 | OK
# 12 | 1 1 0 0 | 1 | 1 0 | OK
# 13 | 1 1 0 1 | 0 | 0 1 | OK
# 14 | 1 1 1 0 | 0 | 0 1 | OK
# 15 | 1 1 1 1 | 0 | 0 1 | OK
#
# =====
#  FIM DA SIMULACAO
#
# ** Note: $stop : C:/ENGG56/Sinaleira/Semaforo.v(50)
# Time: 160 ns Iteration: 0 Instance: /tb_sinaleira
# Break in Module tb_sinaleira at C:/ENGG56/Sinaleira/Semaforo.v line 5
```

Figura 40 - Saída do console com validação caso-a-caso

4. CONCLUSÃO

Ao longo do desenvolvimento deste trabalho foi possível consolidar, na prática, diversos conceitos fundamentais de projeto de circuitos digitais apresentados ao longo da disciplina. A implementação dos módulos em Verilog, aliada às etapas de simulação funcional e análise temporal, permitiu uma compreensão mais profunda da relação entre especificação teórica, descrição em HDL e comportamento real do circuito após a síntese. Cada projeto proposto apresentou desafios específicos, exigindo atenção tanto à lógica de controle quanto ao correto sincronismo entre sinais.

Durante a execução das atividades, dificuldades relacionadas à modelagem de máquinas de estados, ao controle de tempos de leitura e escrita em memória e à garantia de comportamento determinístico foram gradualmente superadas por meio de testes sistemáticos e refinamento do código. O uso de testbenches bem estruturados mostrou-se essencial para a validação das funcionalidades esperadas, além de facilitar a identificação de erros sutis que poderiam comprometer o funcionamento global dos sistemas.

De forma geral, o trabalho evidenciou a importância de boas práticas de codificação em Verilog, como a escrita clara de processos sequenciais e combinacionais, o cuidado para evitar inferências indesejadas de hardware e a validação contínua por simulação. A experiência adquirida contribui diretamente para a formação dos integrantes do grupo, preparando-os para o desenvolvimento de projetos digitais mais complexos e próximos de aplicações reais, reforçando o papel da verificação e da análise temporal como etapas indispensáveis no projeto de circuitos integrados digitais.

5. ANEXOS

Todos os arquivos de implementação desenvolvidos neste projeto — incluindo os códigos Verilog dos módulos RemoteController, Acumulador, FSM, TOP, FSM_Control e do semáforo, bem como os respectivos testbenches, scripts de restrições temporais e arquivos de configuração — estão disponíveis publicamente em um repositório no GitHub.

O repositório organiza os projetos por diretórios, contendo também os arquivos de pinagem para a placa DE2-115, logs de simulação, memórias de inicialização e documentação complementar. Essa estrutura permite a reprodução completa dos experimentos, a verificação independente dos resultados apresentados e a adaptação do código para outras plataformas ou aplicações.

O link para o repositório é: <https://github.com/Eduardo-vitors/PCID>

6. REFERÊNCIAS

INTEL CORPORATION. **Quartus Prime Lite Edition User Guide**. Santa Clara, CA: Intel Corporation, 2020. Disponível em: <https://www.intel.com>. Acesso em: jan. 2026.

INTEL CORPORATION. **TimeQuest Timing Analyzer User Guide**. Santa Clara, CA: Intel Corporation, 2020. Disponível em: <https://www.intel.com>. Acesso em: jan. 2026.

MENTOR GRAPHICS. **ModelSim User's Manual**. Wilsonville, OR: Siemens EDA, 2019. Disponível em: <https://eda.sw.siemens.com>. Acesso em: jan. 2026.

NEC ELECTRONICS. **NEC Infrared Transmission Protocol Specification**. Application Note. Tokyo: NEC Corporation, s.d.

OUROBOROS SISTEMAS EMBARCADOS. *Infravermelho: decodificando protocolo de controle IR || STM32*. YouTube, s.d. Disponível em: <https://www.youtube.com/watch?v=gD9imRCv2lA>. Acesso em: jan. 2026.